Luma Development Reference

Table of contents

About Luma 8

Collaborative Development Guideline 9
- About the Three-Stream Approach . . . 9
  - Steps of the Three-Stream Approach . . . 10
- The "Braindump" . . . 11
- Version Control . . . 12
  - Repositories . . . 12
  - Branches . . . 13
  - Pull-Requests (PRs) . . . 13
  - Contributor's Roles . . . 14
  - Security and Secrets . . . 14
  - Licensing . . . 14
  - Releases . . . 15
- The Development Reference . . . 15
  - Writing Guidelines for the Development Reference . . . 15
- Python Method Writing Convention . . . 16

Testing Guideline for Developers 17

I General Development Settings 18

Python Environment Setup 19
1. Overview . . . 19
2. Goals . . . 19
3. Prerequisites . . . 20
4. Output . . . 20
5. Process . . . 20
   - Step 1 - Create an isolated environment . . . 20
   - Step 2 - Install Luma-GE from GitHub . . . 21
   - Step 3 - Quick verification . . . 21
   - Step 4 - Tests (coming soon) . . . 21
6. Change management (three-stream approach) . . . 22
7. References . . . 22

Earth Engine Initialization and Authentication 23
1. Overview . . . 23
2. Prerequisites . . . 23
3. Output . . . 24
4. Steps . . . 24
4a. Manual authentication (interactive OAuth) . . . 24
4b. Service account authentication (deployment and CI) . . . 24
5. Troubleshooting . . . 25
5.1 Authentication fails . . . 25
5.2 Quota / project errors . . . 26
5.3 Exports fail . . . 26
6. References . . . 27

Helpers 28

Overview 29

Input 30

Output 31

Process 32
Earth Engine Export Utilities . . . 32
Export Parameters . . . 32
Export Process . . . 33
Export Task Monitoring . . . 34

II Luma Geospatial Engine 35

1 Acquisition of Near-Cloud-Free Satellite Imagery 36

Module Overview 37

Input 38

Output 39

Process 40
1.1 Selection of AOI . . . 40
1.1.1 AOI from Shapefile . . . 40
1.1.2 AOI from Indonesia Administrative Boundaries . . . 40
1.2 Correction of Shapefile Geometries . . . 41
1.2.1 Correction of Shapefile Geometries . . . 41

1.3 Selection and Preparation of Satellite Imagery . . . 43
1.3.1 Satellite Data Acquisition and Filtering . . . 43
1.3.2 Sentinel-2 Spatial Resolution Harmonization . . . 46
1.3.3 Topographic Correction . . . 47
1.4 Mosaic and Composite for Satellite Imagery . . . 48
1.5 Visualization and Saving of Processed Satellite Imagery . . . 49
1.5.1 Visualization and Saving Processed Satellite Imagery . . . 49

2 LULC Classification Scheme **51**

Module Overview **52**

Input **53**

Output **54**

Process **55**
2.1 Selection of Classification Scheme . . . 55
2.1.1 User Uploading Their Own Classification Scheme . . . 55
2.1.2 User Entering the Classification Manually . . . 56
2.1.3 User Selecting a Default Classification Scheme . . . 57
2.2 Saving and Downloading the LULC Classification Table . . . 57
2.2.1 Saving and Downloading the LULC Classification Table . . . 57

3 Sample Data Generation **59**

Module Overview **60**

Input **61**

Output **62**

Process **63**
3.1 User Uploads Their Own Class-Labelled Sample Data . . . 63
3.1.1 Verifying Sample Data from User's Input . . . 63
3.2 User Use Default Classification Scheme . . . 65
3.2.1 Probability map generation using modular mapping approach . . . 65

4 Sample Data Quality Analysis **66**

Module Overview **67**

Input **68**

Output **69**

Process **70**
4.1 Specifying the Separability Analysis Parameter . . . 70
4.2 Conducting the Separability Analysis . . . 70
4.2.1 Initialize the Separability Analysis . . . 70
4.2.2 Validating the sample data's geometry . . . 70
4.2.3 Converting sample data into Earth Engine format . . . 71
4.2.4 Starting the separability analysis computation . . . 71
4.2.5 Counting sample data statistics . . . 71
4.2.6 Extracting spectral and predictor value . . . 72
4.2.7 Calculating the extracted spectral feature statistics . . . 72
4.2.8 Calculating separability index . . . 72
4.3 Visualization of Reference Data's Spectral Profile . . . 73

5 Predictor Generation **75**

Module Overview **76**

Input **77**

Output **78**

Process **79**
5.1 Predictor Availability Check and Selection . . . 79
5.2 Predictor Computation . . . 80
5.3 Predictors Visualization and Analysis . . . 84
5.4 Semi Automatic Predictor Selection Optimization . . . 85

6 LULC Map Generation **89**

Module Overview **90**

Input **91**

Output **92**

Process **93**
6.1 Extracting spectral feature from sample data . . . 93
6.1.1 Splitting the sample data . . . 93
6.2 Creating classification model . . . 93
6.2.1 Creating classification model . . . 93
6.2.2 Reviewing classification model's quality . . . 94
6.2.3 Visualizing LULC classification map result . . . 94
6.2.4 Downloading the LULC classification map result . . . 94
6.3 Default classification scheme classification from probability layers . . . 95

7 Thematic Accuracy Assessment **96**

Module Overview **97**

Input **98**

Output **99**

Process **100**
7.1 Checking Prerequisites from Previous Modules . . . 100
7.2 Thematic Accuracy Assessment . . . 100
7.2.1 Selecting The Workflow . . . 100
7.2.2 Generate Reference Sample Workflow . . . 100
7.2.3 Accuracy Computation Workflow . . . 101
7.2.4 Starting the thematic accuracy assessment . . . 102
7.2.5 Spatial Distribution of Error . . . 104
7.2.6 Reviewing the thematic accuracy result . . . 105

8 Post Classification Analysis **106**

III Luma Interface **107**

1 Map Generation **108**

Overview **109**

Step 1: Basic Information **110**
Step 1.1: Geographic Area . . . 110
Step 1.1.a: Upload Area . . . 111
Step 1.1.b: Draw Area . . . 114
Step 1.1.c: Administrative Boundaries . . . 116
Step 1.2: Time Period . . . 118
Step 1.3: Satellite Imagery . . . 120
Step 1.4: Confirm and Preview . . . 122

Step 2: Land Use and Land Cover Classification **125**
Step 2.1: Land Use & Land Cover Classification Scheme . . . 126
Step 2.1.a: Own Classification . . . 126
Step 2.1.b: Default Classification . . . 131
Step 2.2: Try Epistem-AI Recommendations . . . 134

Step 3: Mapping Sample Data **135**
Step 3.1: Upload Sample Data . . . 135

Step 3.2: On Screen Sampling . . . 138
Step 3.2.a: Add Point . . . 140
Step 3.2.b: Draw Area . . . 144
Step 3.3: When user used Default Classification in Step 2 . . . 144

Step 4: Customize Mapping Parameters **145**
Step 4.1: Predictor Selection . . . 145
Step 4.2.a: Default Predictors . . . 145
Step 4.2.b: Upload Predictors . . . 147
Step 4.2: Classification Model Training . . . 148
Step 4.3: When user used Default Classification in Step 2 . . . 150

Step 5: Map Generation **152**
Step 5.1: Model Accuracy Assessment . . . 152
Step 5.2: LULC Class Composition . . . 154
Step 5.3: Thematic Accuracy Assessment . . . 155

Additional Features **159**
Feature 1: Save Progress . . . 159
Feature 2: Download Map . . . 161
Feature 3: Share Map . . . 162
Feature 4: Improve Accuracy . . . 163

2 Change Analysis **165**

3 Collaborative Mapping **166**

About Luma

Land Use Mapping for All (Luma) is an online mapping platform that integrates open-source spatial data and cloud-based computing for generating land use and land cover (LULC) maps. Luma is designed to accommodate a wide range of users through its user-friendly interface and guided steps, enabling anyone to easily produce custom LULC maps tailored to their needs.

This work is part of the **Evolving Participatory Information System for Nature-based Climate Solutions (Epistem)** initiative, which aims to develop an open-source landscape monitoring technology that can address multiple thematic requirements of diverse actors and stakeholders of nature-based climate solutions.

Collaborative Development Guideline

About the Three-Stream Approach

- **What is the three-stream approach?**

The three-stream approach is a workflow that separates brainstorming, design, and implementation into clear yet connected streams. Each stage of development has its own dedicated space, allowing for a structured and transparent decision-making and implementation process throughout the development cycle instead of mixing ideas, references, and implementations in scattered places.

- **Why the three-stream approach?**

The approach intends to solve a common problem in collaborative software development: ideas were discussed in one place, decisions were written somewhere else, and code evolved on its own, making it hard to tell what was current or why something changed. Someone working a week later or in a different time zone would often re-solve the same problem or unknowingly work against an outdated assumption.

The three-stream approach keeps context visible through asynchronous communication, so work stays aligned even when people are not in the same place or working at the same time.

- **What are the three streams?**
  - The "Braindump" (on Notion)

    This is where ideas start. We use this Braindump to record all notes and discussions regarding future developments and improvements of the Luma Geospatial Engine. Nothing here is final and things are allowed to change or be discarded.
  - The "Development Reference" (using Quarto)

    This is where ideas become decisions. Content here is structured and stable enough for developers to refer to. It provides shared context so contributors can understand past decisions, even if they were not part of the original discussion.
  - The "Code Implementation" (Luma Geospatial Engine Python package)

    The Luma Geospatial Engine is the final implementation of the development, in the form of a Python-based package that powers the Luma platform.

Both the Development Reference and Code Implementation are hosted on GitHub for version control.

Steps of the Three-Stream Approach

1. Start by writing your comments in the **Braindump** under the relevant section. Tag the person you want to discuss with so your input is visible and does not get missed.
    - If someone disagrees with your comment, add the topic to the **To-Settle** list. Use the page to write down the key arguments so they can be discussed efficiently in a meeting. Once the disagreement is resolved, move the topic to **Work in Progress**.
    - If there is no disagreement, you can add the topic directly to **Work in Progress**.
2. For agreed development work, create a new branch in **luma-dr** and document the proposed changes there.
3. Next, open a pull request in **luma-dr** from your new branch to the main branch. **Pull requests are reviewed during regular pull request meetings, where changes are either accepted or rejected.**
4. Once changes are merged into the main branch of **luma-dr**, contributors can begin implementing them in **luma-stack**. Implementation should always start from a new branch.
5. Submit a pull request on **luma-stack** to propose your changes for review at the next pull request meeting. If needed, also submit a pull request on **luma-dr** to update the documentation describing the changes.
6. During each pull request meeting, the team will:
    - Review and accept pull requests in **luma-stack** that implement previously approved changes
    - Update **luma-dr** based on approved pull requests
    - Review new pull requests in **luma-dr** proposed for future updates

> Tip
>
> For more reference on the workflow (including a case example), refer to this slide presented during the Bootcamp.

The "Braindump"

The Braindump serves as the central hub for refining ideas and discussions related to Luma's development prior to their formal inclusion in the Development Reference. It contains organized pages that ensure alignment with the current Development Reference, along with sections dedicated to other discussion topics.

- **Structured pages for Luma Geospatial Engine development**
  - The content of each page in this section reflects the latest version of the Development Reference (i.e., the `main` branch of luma-dr).
  - Ideas and discussions can be provided by commenting on the related section of each page. **Tag the relevant person in the comment to ensure the comment is read by others.**
  - The comments will be resolved in two cases:
    * No follow-up is needed after the discussion.
    * The follow-up has been listed in the **Work in Progress** or **To-Settle** section.
- **Structured pages for Luma User Interface development**

  This section is a database dedicated to the implementation details of the Luma User Interface.
- **To-Settle**
  - This section is dedicated to listing topics that arise from disagreements among multiple people in the comments on the **Structured pages for Luma Geospatial Engine development** and that require further discussion or a decision before updates are documented in the Development Reference.
  - The disagreement should be briefly described inside the page.
  - Choose an urgency level of the conflict to determine if the meeting should be scheduled as soon as possible.
  - Once the conflict is settled, the issue can either be moved to be listed in the **Work in Progress** or dismissed.
- **Work in Progress**
  - This section is dedicated for tracking ongoing tasks related to Luma Geospatial Engine updates.
  - Each task goes through several stages of development according to the contribution workflow. The stage at which an issue is currently is noted in the **Completion status** column.

- **Unresolved issues from Pull Request approvals**

  This section records issues or feedback that emerged during the pull request review process and need additional work or clarification before approval.

- **Meeting Notes**

  A collection of notes, summaries, and key decisions from meetings related to Luma's development activities.

- **Sketchbooks**

  A space for "sketching out" ideas or conceptual designs before formal documentation. This section is intentionally unstructured to encourage open exploration.

Version Control

Version control is a core component of the Luma ecosystem and is managed through GitHub, which serves as the central platform for tracking both Quarto based Development Reference and Luma Geospatial Engine code implementation. GitHub provides a transparent and traceable environment for managing changes to code and documentation over time. Contributors create a dedicated branch for each feature, fix, or documentation update. Once the work is complete, changes are submitted to the main branch through a pull request. Each pull-request is reviewed and approved before merging to ensure quality control and alignment between the development reference and the geospatial engine.

Repositories

1. There are two main repositories currently operating:

   - luma-stack: This repository is used to host Luma Geospatial Engine Python package. To contribute to this repository, please refer to `README.md` in this repository for detailed instructions on setting up a developer environment and configuring your local machine.
   - luma-dr: This repository hosts Luma Development Reference. Contributions follow this workflow:
     1. Clone the repository to your local machine.
     2. Create a new branch for your changes.
     3. Edit the relevant `.qmd` files to implement your updates.
     4. Document your modifications in the Braindump under the "Work in Progress" section.
     5. Test your changes by rendering the Quarto book locally to ensure correctness.
     6. Commit and push your changes to your branch.

7. Submit a pull request for review. All changes must be merged via pull request.
8. Proposed changes will be discussed, and once approved, merged into the main branch. Your feature branch will then be deleted.

2. If a new repository is needed (e.g., for an experimental module, plugin, or related service), please consult with an Epistem admin before proceeding.

Branches

1. Every repository should at least have one `main` branch. This branch represents the most up-to-date and stable version of the project.
2. A branch protection is targeted for the `main` branch. This means contributors cannot directly make changes to the `main` branch. Instead, all updates must go through a pull request (PR) process to ensure the code is reviewed and tested before merging. The branch protection rules are as the following:
   - Require a pull-request before merging (please refer to the **Pull-Requests (PRs)** section in this page for more details)
   - Require at least one approval before merging.
3. Each contributor should create a personal working branch from the `main` branch when developing new features or fixes. Once the work is complete, open a pull request to propose merging those changes into the `main` branch, unless there are circumstances where new branch can be created from another feature branch if the work is closely related.
4. Branch naming convention to specify the specific type of a contribution should be further discussed (e.g., `feature/`, `bugfix/`)

Pull-Requests (PRs)

1. All contributions should be submitted through a pull request (PR) from your own branch. A PR is a formal request for your changes to be reviewed and merged into the `main` branch.
2. Keep a PR focused on a specific topic. This specific topic should also correspond to a task in the **Work in Progress** list in Notion. To help identify the task, name the PR title accordingly with what is written in the **Work in Progress** task list.
3. *Only for `luma-stack` and `luma-dr` repository*: when an update in one repository depends on or affects the other, link the related PRs in their descriptions. For example, if a `luma-stack` update requires changes in `luma-dr`, include a link to the corresponding PR in each repository to maintain traceability.

4. As required by the branch protection rules, every PR must receive at least one approval before merging. This approval should come from an Epistem admin. You may also request additional reviewers for feedback.
5. Reviewers should check for correctness, clarity, performance, and proper documentation. Use GitHub's review options:
    - Comment: suggest improvements
    - Request changes: block merge until issues are resolved
    - Approve: accept the PR as ready to merge
6. The PR title should reflect the main change made. This title will be used as the commit message when the PR is merged into the main branch.

Contributor's Roles

1. Only members of the Epistem consortium are authorized contributors to these repositories.
2. There are three different roles set in the epistem-io organization, each with different levels of access:
    - Admin: has full control of repositories (e.g., create/delete repositories, manage access, merge pull requests).
    - Manage: can review and approve contributions, manage issues, and organize repository settings.
    - Write: can create branches, push code, and submit pull requests for review.

    Contributors should know their assigned role before making any changes to ensure proper permissions are in place.

Security and Secrets

Never commit secrets such as passwords, access tokens, or API keys. Add `.gitignore` file to prevent private files from being accidentally uploaded.

Licensing

To be developed. A standardized licensing framework will be provided to ensure consistent open-source and internal usage across repositories.

Releases

To be developed. The release process defines how stable versions of Luma tools and documentation are packaged, tagged, and shared. Releases ensure that users and collaborators can always access a verified version of the software or documentation.

The Development Reference

The development reference is dedicated as a platform for mainly two purposes:

1. **Formalized the decision made in the Braindump.** All discussions' results that were made in the Braindump are documented accordingly, so that all developments made for the Code Implementation are well-tracked.
2. **Facilitate the development of Luma's technical documentation for end-users.** With the use of the development reference as a structured documentation of Luma's development, this documentation can also be a starting point to create Luma's technical documentation for end-users in the future, with reduced effort.

Writing Guidelines for the Development Reference

- The development reference includes several pages, each serving a different thematic purpose in Luma technology
- Each of the pages is written in the following structure:

    1. **Title of the page**
    2. **Input**: a list of input data needed for the related process in the related page. The inputs are divided into:
        - User's Input: list of inputs provided by Luma's users.
        - Input from External Sources: list of inputs received from sources other than Luma's user.
        - Input from Other Modules: list of inputs from other modules in Luma.
    3. **Output**: list of expected output from the process defined in the page.
    4. **Process**: a detailed step-by-step the execution logic of the functionality described on the page. The process should be defined into the following sub-headings:
        - **Step**: a step represents a distinct change in system state or responsibility. A new step must be introduced when:
            * ownership shifts (e.g. from user action to system execution, or between modules), or
            * a new input set is consumed, or
            * an intermediate output is produced that is required by subsequent logic.

  - **Sub-step**: a sub-step represents an atomic action required to complete a step. Sub-steps must not introduce new inputs or outputs beyond those defined for the parent step.
    * **Luma User Journey**: the description of the processes that occur in the user interface. This part is equivalent to the "User Journey" in the Visio workflow diagram.
      1. An error notification should be written in the `callout-important` block type with the title "Error Handling Notification".
      2. A success notification should be written in the `callout-note` block type with the title "Success Notification".
    * **Luma Geospatial Engine**: the description of the processes that occur in the back-end part. This part is equivalent to the "System Response" in the Visio workflow diagram.
      1. The defined function of the related process in the sub-step should be written in the `callout-tip` block type with the title "Related Function".

- Write the Development Reference using clear and easy-to-understand wording that can be understood by audiences with different levels of technical expertise. Avoid technical jargon where possible while still ensuring that all critical elements of the process are clearly and sufficiently described.

> **!** Important
>
> When writing descriptions of a process, make sure to include all the crucial steps that need to be taken in the related process.

Python Method Writing Convention

To be developed

Testing Guideline for Developers

Part I

General Development Settings

Python Environment Setup

1. Overview

Luma stands for Land Use Mapping for All. The geospatial functionalities that powers Luma is called Luma-GE (Luma Geospatial Engine). Luma-GE is shipped as a Python package because it needs to run in more than one context. The same geospatial engine must be importable by an interface layer (for example, a Streamlit-based UI/UX) and also be usable directly from a notebook for development, debugging, and method exploration. Packaging turns "a folder of scripts" into a reliable, versioned engine with a stable import path (`import luma_ge`), repeatable installs across laptops and CI, and a single place to declare runtime requirements.

Luma-GE uses `pyproject.toml` as the source of truth for packaging and installation. This file must live at the repository root. Modern Python tooling (including `pip`) reads `pyproject.toml` to understand the package identity, its required Python version, and the dependencies that must be installed alongside the engine. In practical terms, the `[project]` section is where we define the package name and version, set `requires-python`, and list the dependencies needed by the functions and methods provided by Luma-GE.

> ⚠ Warning
>
> Dependencies will be kept **parsimonious**. Only add a dependency when a function or method genuinely needs it. This keeps installs faster, reduces conflicts, and avoids dragging heavy geospatial binaries into environments that don't need them.

---

2. Goals

The goal is a developer setup that works the same way on different laptops and in CI. Installs should be standard and boring, and a new contributor should be able to get a working environment quickly without any one-off hacks.

---

3. Prerequisites

You need Git installed because `pip` installs from a Git URL by calling `git` under the hood. You also need a Python environment manager. For geospatial stacks, we recommend Miniforge or Mambaforge (Conda-compatible and lightweight) because many geospatial dependencies are easier to install as pre-built binaries than to compile locally.

To confirm Git and Conda/Mamba are available in your shell, run:

```
git --version
conda --version
```

If you are on Windows, avoid adding Python or Conda to your system PATH unless you know exactly why you are doing it. PATH conflicts are a common source of broken environments.

---

4. Output

A working development environment that can run Luma-GE via an interface or via a notebook, and can also build package artefacts when needed.

---

5. Process

Step 1 - Create an isolated environment

Create a dedicated environment for Luma so your system Python stays untouched and dependency conflicts are contained.

```
mamba create -n luma python=3.11
mamba activate luma
```

If you use `conda` instead of `mamba`, the commands are the same (just slower).

Step 2 - Install Luma-GE from GitHub

If you want a quick install of the latest code without cloning the repo, install directly from GitHub. By default we install from the main branch.

```
python -m pip install "luma_ge @ git+https://github.com/epistem-io/luma-stack.git"
```

If you need a specific branch (for example, to test a feature branch or reproduce a bug), pin it explicitly:

```
python -m pip install "luma_ge @ git+https://github.com/epistem-io/luma-stack.git@<branch-nar
```

> **!** Important
>
> Installing from a Git URL requires `git` to be installed and available on your PATH.

Step 3 - Quick verification

Confirm the package imports correctly.

```
python -c "import luma_ge; print('luma_ge import OK')"
```

If you want a concrete example of how Luma-GE functions and methods are used in practice (outside the interface layer), review the implementation notebook in the repository:

`notebooks/Module_implementation.ipynb`

```
https://github.com/epistem-io/luma-stack/blob/main/notebooks/Module_implementation.ipynb
```

Step 4 - Tests (coming soon)

> **⚠** Warning
>
> **Coming soon**
> A PyTest suite is planned but not implemented yet.
> When it lands, this section will document the default test command (`pytest -q`), where tests live (`tests/`), naming (`test_*.py`), and recommended patterns (fixtures, `monkeypatch`, and mocking for cloud APIs).

---

6. Change management (three-stream approach)

Python environment changes are high-impact when they change the supported Python version, alter core dependency groups (especially geospatial binaries), modify install commands or extras, or change CI behaviour. These changes should follow the three-stream workflow (Notion → luma-dr → luma-stack) so that the documentation and implementation stay in lock-step.

---

7. References

- UBC Geography (2024) *Getting Started with Conda, Virtual Environments, and Python*. Available at: https://ubc-geography.github.io/computing-resources/development-environments/conda-getting-started.html
- pyOpenSci (2025) *Python Package Structure & Layout*. pyOpenSci Python Package Guide. Available at: https://www.pyopensci.org/python-package-guide/package-structure-code/python-package-structure.html
- PyPA (2025) *VCS Support*. Available at: https://pip.pypa.io/en/stable/topics/vcs-support/
- PyPA (2025) *Writing your pyproject.toml*. Python Packaging User Guide. Available at: https://packaging.python.org/en/latest/guides/writing-pyproject-toml/

Earth Engine Initialization and Authentication

1. Overview

Luma-GE uses the Google Earth Engine (GEE) Python API for cloud-based geospatial processing. Before any module can call Earth Engine, the runtime must be authorised.

In practice, "authorisation" is two things glued together:

First, you authenticate as an identity (a human Google account, or a service account).

Second, Earth Engine needs a Google Cloud project context so quotas and usage accounting are applied to the right place. Recent Earth Engine Python client changes have made this project context effectively mandatory for common authentication modes. If Earth Engine cannot determine a "quota project", requests will fail.

Luma-GE currently supports two working authorisation options, each with clear trade-offs:

1. Manual authentication (interactive OAuth) is best for local development and workflows tied to a human identity. It is interactive (browser + copy/paste code) and is not suitable for unattended deployments.
2. Service account authentication is the default for deployments and CI. It is non-interactive and repeatable, but it has no option for exporting to a user's personal Google Drive.

2. Prerequisites

To use Earth Engine from Python, you need:

A Google account with Earth Engine access, and a Google Cloud project that is registered for Earth Engine and has the Earth Engine API enabled.

If you are using service account mode, you also need a service account created in that Cloud project and a JSON private key for it.

For service accounts, ensure the account has the correct IAM permissions on the project. In practice, the Earth Engine docs point you at Earth Engine resource roles, and some auth modes also require permissions related to Service Usage.

3. Output

An authorised session allowing Luma-GE functions to execute Earth Engine computations, traceable to a specific Google Cloud project.

4. Steps

4a. Manual authentication (interactive OAuth)

Manual authentication is the simplest path for developers working locally. It authenticates you as your own Google account. That matters when you need behaviour tied to a human or organisation identity, such as exporting to your Google Drive.

In practice, you run an auth function, a browser window opens, you sign in with a Google account that has Earth Engine access, and you paste an authorisation code back into the terminal (or a notebook cell). After that, initialise Earth Engine with an explicit project ID so usage is attributed correctly.

In Luma-GE, this is implemented in `luma_ge/ee_config.py:authenticate_manually()`, which calls `ee.Authenticate()` followed by `ee.Initialize(project='YOUR_PROJECT_ID')`.

> **Warning**
>
> Luma-GE does not support running manual authentication inside a Streamlit interface. Earth Engine's interactive flow expects terminal/browser control that a hosted app generally does not have.

4b. Service account authentication (deployment and CI)

A service account is a non-human Google identity intended for machine-to-machine access. This is the recommended mode for deployments, containers, scheduled jobs, and CI pipelines.

To create and download a service account key, use Google Cloud IAM: create a service account under your project, then create a JSON key for it. Treat the JSON key like a password. If it leaks, rotate it.

In Luma-GE, service account auth is implemented in `luma_ge/ee_config.py:initialize_with_service_accou`
The implementation loads the JSON key, sets `GOOGLE_APPLICATION_CREDENTIALS`, and initialises Earth Engine. If you do not pass a project ID explicitly, it attempts to infer it from the JSON (`project_id`).

Where should the key live? Anywhere except Git.

For local development, keep the JSON file outside the repo and point to it using `GOOGLE_APPLICATION_CREDENTIALS`, or store it in a local `auth/` directory that is excluded by `.gitignore`.

For deployments, inject the key via environment variables or a secrets manager. Luma-GE's credential discovery helper (`luma_ge/__init__.py:_find_service_account_file()`) searches in this priority order: `GOOGLE_SERVICE_ACCOUNT_JSON_B64`, then `GOOGLE_SERVICE_ACCOUNT_JSON`, then `GOOGLE_APPLICATION_CREDENTIALS`, before looking for local `auth/` folders and container fallbacks.

> **!** Important
>
> Never commit a service account key to GitHub. Public repos are scraped, history is hard to erase, and forks/caches persist. If a key is exposed, rotate it immediately.

Exports are the main bottleneck with service accounts. Earth Engine supports Drive exports via `Export.image.toDrive`, but Drive exports target the authenticated identity's Drive.

A recent Google Workspace policy change adds a very specific sharp edge. From 15 April 2025, newly created Google Cloud IAM service accounts no longer receive the default Google Workspace storage and cannot own Drive items. In practical terms, a service account created after that date will generally fail to export to "My Drive", because there is no Drive storage space for it to own the exported file.

For service accounts, the recommended pathways are exporting to Google Cloud Storage (GCS) or to an Earth Engine Asset, then sharing/copying results as needed. If you must land results in Drive, a more robust pattern is exporting into a Shared Drive, where the Shared Drive owns the file rather than the service account.

If you want a direct download link for quick debugging or open access downloads (without user login), `ee.Image.getDownloadURL()` is available but intentionally limited (maximum request size 32 MB and maximum grid dimension 10,000 pixels). It will not work for large rasters.

5. Troubleshooting

Most authorisation failures fall into a handful of predictable buckets.

5.1 Authentication fails

If Earth Engine says you are not authenticated, you either have no cached user credentials (manual mode) or the service account key was not found/loaded (service account mode). In Luma-GE, `get_auth_status()` is the quickest sanity check because it performs a trivial request to confirm the session can execute.

If manual authentication suddenly starts failing after a library update (especially in notebook-style flows), check the Earth Engine Python client library release notes. A common error message asks you to grant `roles/serviceusage.serviceUsageConsumer` (or the underlying `serviceusage.services.use` permission) on the relevant project.

If a service account works on one developer laptop but fails on another, it is usually one of these: the key file path is wrong or unreadable; the project is not registered for Earth Engine access; the Earth Engine API is not enabled; the service account does not have the required IAM roles; or permissions have not yet propagated.

5.2 Quota / project errors

If you see an error about a quota project not being set, Earth Engine cannot determine which Cloud project should be charged for requests. The Earth Engine guide recommends setting the project explicitly.

Ensure that (a) the project is registered for Earth Engine access, (b) the Earth Engine API is enabled in Cloud Console, and (c) you initialise with an explicit project in code: `ee.Initialize(project='YOUR_PROJECT_ID')`.

If you are using Application Default Credentials, the Earth Engine guide also suggests running `earthengine set_project YOUR_PROJECT_ID` or `gcloud auth application-default set-quota-project YOUR_PROJECT_ID`.

5.3 Exports fail

Export failures are a common trap. `ee.Image.getDownloadURL()` is intentionally limited to small downloads (maximum request size 32 MB and maximum grid dimension 10,000). If you exceed those limits, switch to batch exports.

For service accounts, it is suggested exporting to Cloud Storage (`Export.image.toCloudStorage`) or to an Earth Engine Asset. For large exports into a user's personal Drive, use manual authentication.

To understand limits and constraints applied to a project (task concurrency, storage, request rates, and other quotas), use the Earth Engine quotas and monitoring documentation, plus Cloud Console quota and monitoring pages.

6. References

Google for Developers (2025) *Authentication and Initialization*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/auth

Google for Developers (2025) *Earth Engine access*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/access

Google for Developers (2025) *Service Accounts*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/service_account

Google for Developers (2024) *ee.Image.getDownloadURL*. Earth Engine API reference. Available at: https://developers.google.com/earth-engine/apidocs/ee-image-getdownloadurl

Google for Developers (2025) *Export.image.toDrive*. Earth Engine API reference. Available at: https://developers.google.com/earth-engine/apidocs/export-image-todrive

Google for Developers (2025) *Export.image.toCloudStorage*. Earth Engine API reference. Available at: https://developers.google.com/earth-engine/apidocs/export-image-tocloudstorage

Google for Developers (2025) *Earth Engine Python client library release notes*. Available at: https://developers.google.com/earth-engine/docs/python-client-lib/release-notes

Google for Developers (2025) *Earth Engine quotas*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/usage

Google for Developers (2025) *Monitoring usage*. Google Earth Engine Guides. Available at: https://developers.google.com/earth-engine/guides/monitoring_usage

Google Cloud (2025) *Set the quota project*. Google Cloud documentation. Available at: https://docs.cloud.google.com/docs/quotas/set-quota-project

Helpers

Overview

The helper module is design to handle features or operations that will be utilized by several modules. Currently, the module handle data output specially raster based data. This operations is utilized by module 1, 5, and 6. Furthermore, there are several options for data output namely, direct download, google drive download, and cloud storage export. The complex export options are wrapped by helpers module for ease integration with front end code

Input

Name of Input	Input Type	Details
Earth Engine image (ee.Image)	Module output	magery composite (module 1), predictors (module 5), and classification image (module 6)
Export parameters	User's input	File name, destination, projection, and scale (pixel size)

Output

Raster based data (GeoTiff)

Process

Earth Engine Export Utilities

Export Parameters

Prior to export any earth engine image, the user must define several parameters required for export

Luma User Journey

1. The user select which export option they are going to use
2. The user define the required parameters for export. These parameters are as follows:
    1. File name (default file name is provided according to each module output)
    2. Spatial resolution (default to 30 m, since Landsat mission is the only one supported)
    3. Coordinate Reference System (CRS). Use WGS 1984 (EPSG:4326) as the default value. If the user wish to use different CRS, they must specified the EPSG code (i.e *'EPSG:4326'*, *'ESRI:54009'*, or *'SR-ORG:6864'*)
    4. For module 5, the user select which predictors to be exported. The predictor can be selected one by one, stacked of predictors, or by category

> Feature Note
>
> Current version of Luma GE only support direct download, which limits the data size to 32 MB. Google Cloud Storage (GCS) and google drive download are not supported in this phase.

Luma Geospatial Engine

1. The system creates a predefined default names according to the module and dataset exported.
2. Extract the geometry from the selected AOI, as prerequisites for export
3. Validate scale and CRS and raise a helpful error if invalid scale or CRS is detected

Related Function

`helper.EE_Export_Manager.santize_export_name()`: Standardize user input file name according to the system need. Replace invalid characters for GCS, Drive, and local file system
`helper.EE_Export_Manager.extract_geometry()`: Extract the geometry from `ee.featureCollection`(AOI) for export purpose
`helper.EE_Export_Manager.validate_scale()`: Validate user input pixel size value, design to avoid negative value what might cause the system to break. Default value use 30 m
`helper.EE_Export_Manager.validate_crs()` : Perform semantic validation for user input CRS code. The CRS content itself is not validated. Default values is WGS 1984 (EPSG:4326)
`helper.EE_Export_Manager.validate_image()` : Validate if the object pass to export manager is an `ee.image`
`helper.EE_Export_Manager.validate_export_params()` : validate the entire export parameters, raise helpful errors if detected

Export Process

Direct Download

Luma Geospatial Engine

1. The direct download function in Luma-GE used Earth Engine's ee.Image.getDownload-URL function, which generate a link for small chunks of image data in GeoTIFF or NumPy format. Maximum request size is 32 MB, maximum grid dimension is 10000.
2. The parameters used for the export is passed from user input

Related Function

`helper.EE_Export_Manager._export_direct_download` : Generate a data download link for direct download option
`helper.EE_Export_Manager.export_image` : Function to initialize earth engine export with input parameters validation while supporting several export option

Google Cloud Storage Export

To be developed

Google Drive Export

To be developed

Export Task Monitoring

Luma User Journey

1. The user can monitor the status of their export directly in Luma
2. If there's no update within 1 minute, they can refresh the task monitor to acquired the latest update of current task status
3. The user receive feedback regarding the status of the export task (in progress, complete, failed, etc)

Luma Geospatial Engine

Related Function

`helper.EE_Export_Manager.get_task_status()`: Return task status using cache to reduce API calls
`helper.EE_Export_Manager.get_active_task()`: Return activate task information, especially the "ready" and "running" state
`helper.EE_Export_Manager.remove_task()`: Remove task from local tracking without affecting the task in earth engine task monitor
`helper.EE_Export_Manager.remove_completed_task()`: Remove all completed task for future monitoring. The removed task include all of the terminal states (complete, failed, or canceled export task)

Part II

Luma Geospatial Engine

1 Acquisition of Near-Cloud-Free Satellite Imagery

Module Overview

The output of this module is near–cloud-free satellite imagery that corresponds to the user-defined area of interest, which is essential for generating accurate and reliable land use and land cover (LULC) datasets. This module emphasizes the generation of pre-processed and corrected satellite imagery, incorporating procedures such as cloud masking, band and value standardization, as well as compositing.

Input

Name of Input	Input Type	Details
Area of Interest	User's input	
Target map date	User's input	YYYY or DD-MM-YYYY
Desired spatial resolution	User's input	in meters
Maximum allowable cloud cover	User's input	in percentage
Raw satellite imagery	System's input	
Area of Interest from administrative boundaries	System's input	

Output

1. Selected AOI.
2. Near-cloud-free satellite imagery.

Process

1.1 Selection of AOI

1.1.1 AOI from Shapefile

This sub-step does not involve any operations within the Luma Geospatial Engine.

1.1.2 AOI from Indonesia Administrative Boundaries

In addition to user define upload the system supports the selection of AOI from Indonesia administrative boundaries at regency level.

1. Load `ee.FeatureCollection` containing Indonesian administrative boundaries from Badan Informasi Geospasial. The data should be uploaded as GEE asset and the link for it must be provided and shared publicly
2. The system extracts the regency name and provide complete list of the available regencies for the user
3. Extract the geometry of the selected regency boundaries and use them as criteria in the imagery filtering
4. Convert the `ee.FeatureCollection` to `geodataframe` and visualize in the map canvas

> Related Function
>
> `input_utils.load_regency_asset:` Retrieve `ee.FeatureCollection` of Indonesia regency/city level administrative boundaries from earth engine asset
> `input_utils.get_regency_name`: Creating a list of regency name for the user
> `input_utils.get_regency_geometry:` Get geometry for the selected regency
> `input_utils.convert_gee_features_to_gdf:` convert `ee.FeatureCollection` to `geodataframe` for visualization in the system

1.2 Correction of Shapefile Geometries

1.2.1 Correction of Shapefile Geometries

1. The system performs a multi-step geometry sanitization process before converting any shapefile or GeoDataFrame into Earth Engine objects. **These input validation processes for shapefiles are designed mainly for Streamlit compatibility**. Each correction is handled by exactly one dedicated function:

   - The Coordinate Reference System (CRS) is automatically corrected and reprojected to WGS 1984 (EPSG:4326).

   > Related Functions
   >
   > `input_utils.shapefile_validator.fix_crs()`: to resolve coordinate reference system issues.

   - Invalid, empty, or topologically broken geometries are detected and repaired.

   > Related Functions
   >
   > `input_utils.shapefile_validator.clean_geometries()`: to repair invalid geometries.

   - An initial structural validation and basic geometry fixing.

   > Related Functions
   >
   > `input_utils.shapefile_validator.validate_and_fix_geometry()`:to validate and fix geometries issues.

   - Polygon-specific validation, including removal of invalid coordinates and simplification of overly complex shapes.

   > Related Functions
   >
   > `input_utils.shapefile_validator.validate_polygons()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

   - Every individual coordinate is checked to ensure it falls within valid global latitude/longitude ranges.

Related Functions

`input_utils.shapefile_validator.is_valid_coordinate()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

- The number of vertices in each geometry is counted and logged (for diagnostic and simplification decisions).

Related Functions

`input_utils.shapefile_validator.count_vertices()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

- Finally, a comprehensive pre-upload validation is executed to guarantee that every geometry is valid, non-empty, correctly typed, and fully compatible with Earth Engine.

Related Functions

`input_utils.shapefile_validator.final_validation()`: to checks point geometries, remove invalid coordinates, and simplify polygon geometries.

2. After the system succeeded to validate the shapefile, the system converted the GeoDataFrame file format into Earth Engine geometry format.

Related Functions

`input_utils.EE_converter.convert_aoi_gdf()`: to conduct the conversion of `.gdf` input file into `ee.geometry` . Three different conversion approaches was defined in this function:

- Primary method: using `geemap.gdf_to_ee()`
- If primary method fails, proceed to secondary method: manual GeoJSON conversion by union multiple geometries first (`gpd.GeoDataFrame([{'geometry': union_geom}], crs=gdf.crs)`) and convert it to GeoJSON (`gdf.to_json()`). **Only polygon and multi-polygon geometry type is supported in this method**.
- If both primary and secondary method fails, proceed to the last option: bounding box conversion to estimate the rectangular approximation of the AOI (`ee.Geometry.Rectangle([bounds[0], bounds[1], bounds[2], bounds[3]])`).

1.3 Selection and Preparation of Satellite Imagery

1.3.1 Satellite Data Acquisition and Filtering

1. The system specifies Landsat data collections for optical processing.

> Related Functions
>
> `data_acquisition.Reflectance_Data.OPTICAL_DATASETS` : class-level dictionary that act as a central registry for all supported Landsat collection. Each key is a user-facing dataset code, and the value contains the metadata needed to fetch and process that collection. The metadata for each entry are `collection`, `cloud_property`, `type`, `sensor`, and `description`
> `data_acquisition.Reflectance_Data.THERMAL_DATASETS` Similar class level dictionary for Landsat TOA collections, used for `get_thermal_bands` function. This dictionary has similar structure with `OPTICAL_DATASETS`
> `data_acquisition.Reflectance_Data.S2_DATASETS` class-level dictionary for Sentinel-2 image collection, currently only support Level 2A Surface Reflectance collection. This dictionary has similar structure with `OPTICAL_DATASETS`

The system currently supports Analysis Ready Data (ARD) for Landsat and Sentinel-2 satellite mission. The detailed mission and temporal coverage for each satellite data is as follows:

- Landsat 1 Multispectral Scanner/MSS (1972 - 1978)
- Landsat 2 Multispectral Scanner/MSS (1978 - 1982)
- Landsat 3 Multispectral Scanner/MSS (1978 - 1983)
- Landsat 4 Thematic Mapper/TM (1982 - 1993)
- Landsat 5 Thematic Mapper/TM (1984 - 2012)
- Landsat 7 Enhanced Thematic Mapper Plus/ETM+ (1999 - 2021)
- Landsat 8 Operational Land Imager/OLI (2013 - present)
- Landsat 9 Operational Land Imager-2/OLI-2 (2021 - present)
- Sentinel 2A/B Multispectral Instrument/MSI (2015 - present)

> Note
>
> **Feature Warning**
> Sentinel-2 Level 2A only extends back to march 2017, since the European Space Agency (ESA) did not systematically generate Level 2A product until then. Additionally, the 2017-2018 level 2A coverage in the earth engine collection is not yet global.

2. To address the limited coverage of Sentinel-2 in 2015-2017, Luma automatically select the Level 1C ToA reflectance data if the user select those years. Atmospheric correction of this data is not applied due to complexity of sentinel-2 pre-processing pipeline. Atmospheric correction for level 1C data will be applied in the upcoming update.

3. The system retrieves images that match user's selected criteria, including spatial resolution, target year, and cloud cover based on the following input parameters: AOI, target map date, and cloud cover threshold.

> Related Functions
>
> `data_acqusition.parse_date_input()`: Parse date input to YYYY-MM-DD format. Accepts either a year (int or 4-digit string) or a full date string. For year inputs, returns Jan 1 for start dates and Dec 31 for end dates. This function is used by `get_s2_optical_data`, `get_optical_data` and `get_thermal_bands`
> `data_acquisition.Reflectance_Data.get_optical_data()`: to retrieve and the Landsat image collection based on the input parameters. This function uses the `mask_landsat_sr`, `apply_scale_factors`, and `rename_landsat_bands` to return a standardize and ready to use image collection
> `data_acquisition.Reflectance_Data.get_s2_optical_data()`: Retrieve Sentinel 2 image collection based on input parameters provided. This function uses the `mask_s2_clouds`, and `rename_s2_bands` to return a standardize and ready to use image collection

   - For Landsat imagery, cloud masking is applied using the quality assurance band (QA_PIXEL) that encodes pixel conditions as bitwise flags using specific bit flags. Deterministic bit tests are applied to identify and exclude pixels affected by cirrus, clouds and cloud shadows. The bit value is adjusted based on the Landsat sensor selected by the user. Landsat 8-9 QA_PIXEL stored per-pixel confidence levels for clouds cirrus, and clouds shadow. These confidence values (ranging from 0–3) are extracted and filtered using hard-coded thresholds, such that pixels with medium to high confidence are masked out. Other Landsat sensor did not have confidence bit, therefore, only deterministic bit are used for cloud masking

   - For Sentinel-2 imagery, cloud masking is implemented using Cloud Score+ quality assessment processor. This approach use weakly deep learning approach to grade the quality of individual image pixels and assign per-pixel quality assessment scores. This masking approach is conducted using link collection function in GEE, since Cloud Score+ collection have the same id and `system:index` properties as the individual Sentinel-2 image collection. Cloud Score+ produce more accurate cloud masking compared to other cloud masking approach such as s2cloudless and QA60. Methodological description of Cloud Score+ can be found in Pasquarella 2023

> Related Functions
>
> `data_acquisition.Reflectance_Data.mask_landsat_sr()`: Mask clouds, shadows and cirrus for Landsat Collection 2 SR using QA_PIXEL band with confidence threshold
> `data_acquisition.Reflectance_Data.mask_s2_clouds()`: Mask cloudy, hazy, and cirrus-affected pixels in a Sentinel-2 image using Cloud Score+ algorithm

- For Landsat imagery, value scaling is implemented to convert the raw digital numbers (DNs) into physically meaningful surface reflectance values. A scale factor 0.0000275 and offset -0.2 are applied, as defined in the Landsat Collection 2 Surface Reflectance metadata. This conversion standardize the reflectance values to a range of approximately 0 to 1, enabling accurate comparison and analysis across different scenes and sensors.

  > Related Functions
  >
  > `data_acquisition.Reflectance_Data.apply_scale_factors()`: to apply Landsat collection 2 scaling factors using the following formula: Digital Number (DN) * scale_factor + offset. The scale factor will only be applied to optical bands.

- The spectral band is standardized according to the semantic names of the band itself. The common naming convention shared across the luma-ge pipeline, enabling downstream modules to reference bands by semantic name rather than sensor specific identifiers.

  > Related Functions
  >
  > `data_acquisition.Reflectance_Data.rename_landsat_bands()`: to standardize Landsat Surface Reflectance (SR) band names based on sensor type. From 'SR_B*' or 'B*' to 'NIR', 'GREEN', etc.
  > `data_acquisition.Reflectance_Data.rename_s2_bands()`: Rename Sentinel 2 SR bands to standardized semantic names (i.e B5 to RED_EDGE1)

4. The system then use the same parameter in `optical_data` and use it to search collection 2 TOA data and (except for Landsat 1-3) select the thermal band. The selected thermal band is then stacked with multi-spectral band from SR data. Since band 11, contain larger calibration uncertainty, only band 10 is used. No additional pre-processing is conducted for thermal band since digital number value alone is considered satisfactory for discriminating land cover features.

Related Functions

`data_acquisition.Reflectance_Data.has_thermal_capability()`: Function to check if the Landsat mission selected contain thermal bands
`data_acquisition.Reflectance_Data.get_thermal_bands()`: to retrieve thermal band from Landsat collection 2 TOA data

1.3.2 Sentinel-2 Spatial Resolution Harmonization

Sentinel-2 bands have three spatial resolution: 10 m (Blue, Green, Red, NIR), 20 m (Red Edge 1, Red Edge 2, Red Edge, 3, Red Edge 4, SWIR 1, SWIR 2), and 60 m (Aerosol, Water vapor). For the final imagery reported to the user, the 60 m bands will be dropped from the composite, since it did not contain valuable information for classification task. To utilize the spectral richness of sentinel 2 imagery, image sharpening is implemented, transforming the 20 m bands to 10 m. Since Earth Engine did not have built-in pan-sharpening algorithm, High Pass Filter (HPF) approach is used to sharpen the 20 m bands.

HPF is traditional image fusion algorithm based on Multi-Resolution Analysis (MRA) framework. HPF extract high frequency information from higher resolution image and inject them to lower resolution multi-spectral images by specified weights. This approach is chosen since some studies that HPF produce more accurate sharpened image (Du et al, 2016; Zheng et al, 2017). Since Sentinel 2 did not panchromatic band, the average value of visible and near infrared bands can be used a substitute for panchromatic band in pan-sharpening approach (Kaplan, 2019)

Related Functions

`data_acquisition.Reflectance_data.sharpen_s2_bands()`: Perform image sharpening for the 20m Sentinel-2 bands using high pass filter (HPF). HPF sharpening consist of three main steps: (1) build pseudo-panchromatic bands from the average value of VIS-NIR band. (2) Applied Laplacian HPF kernel, perform bilinear resampling, and inject spectral reflectance detail. (3) Stack the sharpen image with VIS-NIR band
`data_acquisition.Reflectance_data.resample_s2_band()`: Alternative approach for sentinel-2 band spatial resolution normalization. This approach only change the pixel size, without improving the spatial resolution of the 20 m bands.

Feature Note

The approach as well as the procedure used to perform spatial harmonization is not shown to the user. The user only receive the harmonize imagery and note regarding the approach used.

> **Feature Warning**
>
> The HPF operations required significant computation time and resource for large area. Sharpening for large area (primary regency level) might hit Earth Engine's user memory limit. If this happen, set the sharpening to false `data_acquisition.final_Image.get_temporal_composite(Sharpen=False):`

1.3.3 Topographic Correction

Topographic correction are additional pre-processing for satellite imagery to mitigate the influence of rugged terrain on surface reflectance data. This pre-processing approach is crucial if there's variation in surface reflectance values between pixels with similar land cover and biophysical-structural properties.

Since Sentinel-2 is already topographically corrected, this option is available for Landsat collections only. Luma-GE applied the a pixel-wise Sun-Canopy-Sensor with a c-correction (SCSc) proposed by Soenen et. al. (2005). This approach use the sun-canopy-sensor (SCS) with a semi-empirical moderator (C) to account for diffuse radiation (Poortinga et. al. 2019). The ALOS 30m Global Surface Model version 4 is used for topographical base.

The SCS method consist of two primary steps, namely illumination condition and illumination correction. The illumination condition (IC) determine the pixel brightness given its slope and orientation. The illumination correction is applied for each spectral band with empirically derived c-value by running a linear regression between reflectance value and IC. However, this correction is applied selectively, according to a mask to only work on pixels in which corrections is meaningful (Poortinga et. al. 2019).

> **Related Function**
>
> `data_acquisition.additional_correction.illumination_condition():` Compute the illumination condition (IC) for a single image and append it as an additional band along with cosZ, cosS, and slope.
> `data_acquisition.additional_correction.illumination_correction():` Perform illumination correction by fitting a linear model between IC and the given band over masked pixels, derive the empirical c-value, and apply the SCSc correction formula.
> `data_acquisition.additional_correction.apply_topo_corr():` Single function wrapper that executes both IC and illumination correction.

1.4 Mosaic and Composite for Satellite Imagery

The final image (single layer) which will be presented to the user is created using `final_image` methods, which resulted in mosaic imagery or temporal aggregation composite imagery.

1. Mosaicking

   Mosaicking approach simply stacks all images in the collection and, for each pixel, selects the value from the topmost image according to the collection's ordering. The system implement quality mosaic, which allows the use of quality band that affected the order of mosaic

   > Related Functions
   >
   > `data_acquisition.final_Image.get_quality_mosaic` create a quality mosaic image based on quality band (source band). `add_quality_band` compute or select the band that will be used for creating the mosaic. Three quality band is supported, namely NDVI and NIR. The mosaic is created based on the highest pixel value of the corresponding band

2. Temporal Aggregation

   Temporal aggregation used statistical reducers, (such as median) to combine pixel values from an image collection over time. A **median composite** often produces a noticeably cleaner image because cloud and other high-reflectance features tend to occupy the upper tail of the distribution. This approach, naturally excludes the feature, resulting in fewer artifacts and a more consistent surface reflectance. However, if at certain location there's no valid pixel across the image collection, the location will resulted in no-data (blank region). Temporal aggregation function is paired with `calculate_coverage` to provide report of the valid pixels in an AOI.

   > Related Functions
   >
   > `data_acquisition.final_Image.get_temporal_composite:` Create a temporal aggregation composite using earth engine reducers. Supported reducers are `median`, `mean`, `min`, `max`, and `percentile`

3. Valid Pixel Reporting

   To provide comprehensive report on the final image creation, system provide the data range which composite is created. Additionally, valid pixel and no data is estimated and reported to the user.

Related Functions

`calculate_coverage` estimate, the valid pixels (unmasked by cloud masking) within AOI. This quantified the 'no data' or blank blocks on the final image. This served as alternative approach in reporting the cloud cover within AOI, since in theory, the cloud cover within AOI is already masked out during cloud masking procedure. This function is used within the `get_temporal_composite` and reported in the final image creation.
The `calculate_coverage` serves as an alternative to calculate cloud cover within AOI. Since cloud cover should be masked out during cloud masking and remove during temporal aggregation

1.5 Visualization and Saving of Processed Satellite Imagery

1.5.1 Visualization and Saving Processed Satellite Imagery

1. The system collects the statistics of the satellite images found.

   Related Functions

   `data_acquisition.Reflectance_Stats.get_collection_statistics()`: to get comprehensive statistics about the gathered image collection.

2. The system provide a preset of commonly used band combinations as well as optimal value range for the imagery. The commonly used band composites define in Luma-GE are as follows:

   1. True Color Composites (RED, GREEN, BLUE)
   2. False Color Infrared Composites (NIR, RED, GREEN)
   3. Short-wave Infrared Composites (SWIR2, NIR, RED)
   4. Land/Water (NIR, SWIR1, RED)

   Related Function

   `data_acquisition.Vis_Params.BAND_COMBINATIONS`: The list of preset band combinations as well as their default value
   `data_acquisition.Vis_Params.get_combinations_names()`: function to fetch band combinations names from the preset
   `data_acquisition.Vis_Params.get_vis_params()`: function to retrieve band

combinations and visualization parameters from the define list

3. The system provide tools for the user to build their own band combinations and modified the imagery minimum/maximum value as well as the gamma value.

> Related Function
>
> `data_acquisition.build_custom_vis_param()`: function to allow the user to build their own band combinations

4. For details on exporting to Google Cloud Storage, Google Drive, and direct download, refer to helper module
5. The system saved the satellite imagery as an object to be loaded in the next modules.

2 LULC Classification Scheme

Module Overview

This module prompt the user to define the list of land cover or land use class which they're going to classified using the Luma platform. Luma supports user-defined classification schemes and default classification or templates for commonly used classification schemes.

Input

Name of input	Input Type	Details
User's LULC list of classes	User's input	Entered with `.csv` file or manually defined on the UI
Other existing LULC list of classes	System's input	RESTORE+ LULC class

Output

LULC list of classes to be applied in the final classification result

Process

> **!** Error Handling Notification
>
> This module cannot be accessed if the system does not detect any saved satellite imagery from Module 1.

2.1 Selection of Classification Scheme

This sub-step does not involve any operations within the Luma Geospatial Engine.

- If the user chooses to upload their own schema, the system proceeds to Section 2.1.1
- If the user chooses to define the classification manually on the UI, the system proceeds to Section 2.1.2
- If the user chooses a default classification scheme, the system proceeds to Section 2.1.3

2.1.1 User Uploading Their Own Classification Scheme

1. The system automatically detects column headers based on common naming conventions for ID, class name, and color code. If the expected headers are not found, the user must manually assign the correct column names.

   > Related Functions
   >
   > `classification_scheme.LULC_scheme_Manager.auto_detect_csv_columns()`: this function automatically detects the column headers. The set for the common naming conventions are currently hardcoded as the following:
   >
   > - For ID: “id”, “classid”, “kode”, “code”
   > - For class name: “classname”, “class”, “kelas”, “name”, “nama”
   > - For color code: “color”, “colorcode”, “warna”, “colour”, “hex”, “palette”

2. The system validates the uploaded `.csv` to ensure IDs are numeric and unique, applies a set of distinct colors if no color code was provided by the user. The user can change the color code manually.

Related Functions

`classification_scheme.LULC_scheme_Manager.process_csv_upload()`: to validate class data by checking ID integrity, assigning colors (`_generate_distinct_colors`), and returning a success status with a summary message.
`classification_scheme.LULC_scheme_Manager._generate_distinct_colors()`: to generate a predefined set of distinct colors for LULC classes (up to 20). If the number of classes exceeds this limit, the method falls back to generating random colors via `_generate_random_colors()`.
`classification_scheme.LULC_scheme_Manager._generate_random_colors()`: to generate random hex color code.
`classification_scheme.LULC_scheme_Manager.finalize_csv_upload()`: to finalize the CSV file by applying any user-assigned colors, saving and sorting the class list, and clearing temporary data.

2.1.2 User Entering the Classification Manually

1. The system validates user-provided class input for numeric ID, non-empty class name, and uniqueness (if adding new classes).

   Related Functions

   `classification_scheme.LULC_scheme_Manager.validate_class_input()`: Validates class ID and name, checks for uniqueness, and returns a validity flag with an error message if invalid.
   `classification_scheme.LULC_scheme_Manager.add_class()`: Adds a new class or updates an existing class, validates inputs, assigns colors, sorts classes, and updates the next available ID.

2. The system normalizes class name inputs through a sequential process: it first applies Unicode normalization using the NFKD form, converts all characters to lowercase, replaces common separators (`_`, `-`) with spaces, removes non-alphanumeric characters, collapses multiple consecutive spaces, and applies light lexical normalization to reduce superficial grammatical differences, such as singular versus plural forms (e.g. “forest” and “forests”). Missing values (`None` or `NA`) are returned as None. Optionally, words within the class name can be sorted alphabetically to enable order-independent matching (e.g. “Forest Mixed” = “Mixed Forest”)

3. Temporary edit mode flags are managed to differentiate between adding and updating classes.

Related Functions

`classification_scheme.LULC_scheme_Manager.edit_class()`
`classification_scheme.LULC_scheme_Manager.delete_class()`
`classification_scheme.LULC_scheme_Manager.cancel_edit()`

2.1.3 User Selecting a Default Classification Scheme

1. The system loads the chosen classification scheme classes.

Related Functions

`classification_scheme.LULC_scheme_Manager.load_default_scheme()`: to load an existing classification scheme from a dataset.
`classification_scheme.LULC_scheme_Manager.get_default_schemes()`: stores the RESTORE+ LULC class in a dictionary.

2. If the user selects a subset of the default classes, the system stores the selection into a variable that will be used for the reclassification process in Module 6.

Related Functions

`classification_scheme.LULC_scheme_Manager.store_classes_of_interest()`: to store the user's class of interest as a variable that will be used for the reclassification process in Module 6.

2.2 Saving and Downloading the LULC Classification Table

2.2.1 Saving and Downloading the LULC Classification Table

1. The system generates a `.csv` file containing the final LULC classification scheme table.

Related Functions

`classification_scheme.LULC_scheme_Manager.get_csv_data()` : to generate a `.csv` file containing the final classification table that has been standardized by `get_dataframe`.
`classification_scheme.LULC_scheme_Manager.get_dataframe()` : to standardize the column names of the LULC classification table to "ID", "Land Cover Class", "Color Palette"

2. The system checks whether the user has defined at least one class.

> Related Functions
>
> `classification_scheme.LULC_scheme_Manager.has_classes()`: to check if any classes are defined.
> `classification_scheme.LULC_scheme_Manager.get_class_count()`: get the information of the number of the defined classes.

3. The system saved the LULC class classification scheme as an object to be loaded in the next modules.

3 Sample Data Generation

Module Overview

To be developed.

Input

Name of input	Input Type	Details
User's sample data	User's Input	
Default sample data	System's Input	RESTORE+ sample data
LULC classification class table	Input from Other Modules	Output from Module 2
AOI	Input from Other Modules	Output from Module 1
Near-cloud free satellite imagery	Input from Other Modules	Output from Module 1

Output

Statistical analyses of the sample data.

Process

3.1 User Uploads Their Own Class-Labelled Sample Data

1. The system verifies the format data input of the `.shp` inside the `.zip` .
2. Once the input format data is validated, the system runs Section 3.1.1
3. The system loads the sample data after verifying the upload
    - Check the initial sample count and conduct sample filtering based on its geometry, making sure all of the sample is located inside the area of interest
    - Handle a large dataset by stratified sampling so that the sample does not exceed 5000 features, and maintain class distribution representation
    - Convert the shapefile into a geodataframe
    - Updates the class field identifier in the training data dictionary.

> Related Function
>
> `sample_data.SyncTrainData.LoadTrainData()`: to load the sample data into the system by conducting several checks

> Feature note
>
> The main issue is that the original stratified sampling algorithm made sequential `.getInfo()` calls for each class label when calculating class sizes and extracting features, which could cause browser crashes or exceed Earth Engine quotas when processing large datasets (5000+ features). The improved implementation consolidates all server-side computations into a single aggregated request, keeping all operations on the server side until the final `.getInfo()` call, thus avoiding excessive client-server communication and quota limitations.

3.1.1 Verifying Sample Data from User's Input

1. The system checks which column in the sample dataset should be used as the class label.

> Related Functions
>
> `sample_data.SyncTrainData.SetClassField()`: Set the class name field for sample data.

2. The system validates class labels in the sample dataset by filtering out class values that do not exist in the classification scheme from Module 2, and updates the sample data and validation report accordingly.

> Related Functions
>
> `sample_data.SyncTrainData.ValidClass()`: Validate classes in sample data.

3. The system checks the number of sample data for each class. The minimum required sample size per class is currently hard-coded at 20.

> Related Functions
>
> `sample_data.SyncTrainData.CheckSufficiency()`: Check if there are sufficient samples per class.

4. The system filters the training samples so only those that spatially overlap the AOI remain, updates validation counts, and records warnings if AOI filtering fails.

> Related Functions
>
> `sample_data.SyncTrainData.FilterTrainAoi()`: Check if there are sufficient samples per class.

5. The system generates a summary table of the user's uploaded sample data. The summary includes:

   - LULC class ID
   - LULC class name
   - Number of sample data of the LULC class
   - Percentage of the sample data count
   - Sufficiency category of the LULC class' sample data

> Related Functions
>
> `sample_data.SyncTrainData.TrainDataRaw()`: Check if there are sufficient sample data per class.

3.2 User Use Default Classification Scheme

This workflow is activated only when the user selects the default classification scheme in Module 2. Its purpose is to enable users without their own training data to generate an LULC map. LULC map generation under the default classification scheme uses a modular mapping approach.

3.2.1 Probability map generation using modular mapping approach

This section describes the process for generating the probability maps required as input for producing an LULC map when the user selects the default classification scheme. This process runs only once and is not repeated each time a user selects the default classification scheme workflow.

1. A default training dataset is generated using stratified random sampling across Indonesia's regions (Sumatra, Kalimantan, Nusa Tenggara, Jawa-Bali, Maluku, Sulawesi, and Papua). Rather than following the conventional approach of labeling training data directly with final LULC classes, each point in this dataset is instead labeled with information on a set of land cover elements, referred to hereafter as "primitives."

2. Each training data point is then assigned a class label using a decision tree ruleset manually constructed by experts. In this decision tree, primitives serve as nodes, and LULC classes serve as leaves.

   > Related Functions
   >
   > A function to assign class label

3. The class-labeled training data is used to train a Random Forest classification model. A one-versus-rest binary classification approach is applied to generate probability maps for each class in the Epistem classification scheme.

   > Related Functions
   >
   > A function to conduct OVR

4. The resulting probability layers are saved as Earth Engine assets and will be process in Chapter 6

4 Sample Data Quality Analysis

Module Overview

This module assesses the spectral separability of LULC classes using sample data and near-cloud-free satellite imagery. Pairwise class separability is quantified using the Transformed Divergence (TD) method based on pixel-level spectral statistics, using only the spectral data from Module 1.

Since TD algorithms are parametric separability metric, non-parametric separability and visualization tools for sample data quality assessment are supported using Silouette Score, Principal Component Analysis (PCA) and t-SNE (t-distributed Stochastic Neighbor Embedding). PCA and t-SNE are data reduction. Both techniques are designed to transform high-dimensional data into a lower-dimensional space. These approach provides visualization for determining the cluster separability between various predictor used in the classification

This module is diagnostic only and does not produce inputs for subsequent modules. Its outputs support interpretation of class confusion and provide justification for feature enhancement in later stages of the workflow.

Input

Name of Input	Input Type	Details
Parameters for the separability analysis	User's input	Contains: • Spatial resolution of the imagery from Module 1 can be adjusted for the separability analysis • Number of maximum pixels which the analysis will run for each pair of classes
Sample dataset	Input from Other Modules	Output from Module 3
Near-cloud free satellite imagery	Input from Other Modules	Output from Module 1

Output

1. Separability analysis result.
2. Spectral profile visualization and text description.

Process

> ❗ Error Handling Notification
>
> This module cannot be accessed if the system is missing the required inputs from the Input from Other Modules.

4.1 Specifying the Separability Analysis Parameter

This sub-step does not involve any operations within the Luma Geospatial Engine.

4.2 Conducting the Separability Analysis

4.2.1 Initialize the Separability Analysis

This sub-step does not involve any operations within the Luma Geospatial Engine.

4.2.2 Validating the sample data's geometry

1. The system validates the geometry of the sample data stored in the system from Module 3 output.

   > 💡 Related Function
   >
   > `input_utils.shapefile_validator.validate_and_fix_geometry()`:to validate and fix geometries issues.

   > ❗ Error Handling Notification
   >
   > An error message is displayed if the system failed to run `validate_and_fix_geometry()`.

4.2.3 Converting sample data into Earth Engine format

1. The system converts sample data from `.gdf` data into Earth Engine Feature Collection format

> Related Function
>
> `input_utils.shapefile_validator.convert_roi_gdf()`: to convert geo-dataframe into EE Feture Collection. The supported multi-geometries type for this conversions are MultiPoint and MultiPolygon.

> Error Handling Notification
>
> An error message is displayed if the system failed to run `convert_roi_gdf()`.

4.2.4 Starting the separability analysis computation

1. The system initializes the Related Function's for separability analysis with the appropriate required input objects for the `class`.

> Related Function
>
> `sample_data_quality.sample_quality()`: a `class` to handle all the workflows for separability analysis.

4.2.5 Counting sample data statistics

1. The system retrieves the basic statistic information of the sample data.

> Related Function
>
> `sample_data_quality.sample_quality.sample_stats()` : to retrieve the sample data's total sample count, unique classes, class-wise sample counts, and class balance, and includes class names when provided.
> `sample_data_quality.sample_quality.get_sample_stats_df()` : to turn the statistic information into a dataframe format.

4.2.6 Extracting spectral and predictor value

1. The system extracts the pixel-level spectral values for each sample data by sampling the composite image at the specified scale. The system also limits the maximum number of pixels per LULC class that will be extracted to avoid memory overload. Additionally, the sample

> Related Function
>
> `sample_data_quality.sample_quality.extract_spectral_values()` : to extract and compile spectral reflectance values for all sample data. The result is a dataframe containing band values for each sample data.
> `sample_data_quality.sample_quality.limit_samples_per_class()` : limits (down-samples) the number of sample data per class whenever the number of available samples exceeds the maximum pixel threshold with random sampling.
> `sample_data_quality.get_feature_column`: Resolve valid numeric feature/predictor columns from the DataFrame.
> `sample_data_quality._stratified_subsample` : Perform stratified subsampling across classes so non-parametric metrics and embeddings execute quickly on large extracted pixel tables.

4.2.7 Calculating the extracted spectral feature statistics

1. The system computes detailed pixel-level statistics for each LULC class (mean, standard deviation, minimum, maximum, median, and sample count) based on the extracted spectral values and formats these results into a structured summary table.

> Related Function
>
> `sample_data_quality.sample_quality.sample_pixel_stats()` : calculates per-class spectral statistics (mean, std, min, max, median, count) for all bands.
> `sample_data_quality.sample_quality.get_sample_pixel_stats_df()` : converts the computed statistics into a dataframe.

4.2.8 Calculating separability index

1. The system computes the class-to-class separability using Transformed Divergence (TD).
2. The system generates a pairwise separability matrix and converts the results into a structured dataframe.

3. The system categorizes the number of good, weak, and poor class pairs as a summary for user.

> Related Function
>
> `sample_data_quality.sample_quality.check_class_separability()`: calculates the pairwise separability values between classes using Transformed Divergence. The choice of using TD as the algorithm is still hard-coded.
> `sample_data_quality.sample_quality.get_separability_df()`: converts separability results into a dataframe with class names and interpretation levels.
> `sample_data_quality.sample_quality.lowest_separability()`: extracts the class pairs with the lowest separability scores.
> `sample_data_quality.sample_quality.sum_separability()`: produces a summary report counting how many class pairs fall under good, weak, or poor separability categories.
> `sample_data_quality.compute_silhouette_analysis`: Compute non-parametric Silhouette Score analysis on standardized predictor features. Evaluates how tightly grouped each class is relative to its nearest neighboring class in [-1.0, +1.0] without assuming Gaussian distributions or inverting covariance matrices.

4.3 Visualization of Reference Data's Spectral Profile

1. The system provides four different visualization options of the spectral distributions and clustering.

> Related Functions
>
> `sample_data_quality.spectral_plotter.plot_histogram()`: generates overlaid interactive histograms of selected bands showing class-wise frequency distributions
> `sample_data_quality.spectral_plotter.plot_boxplot()`: creates interactive boxplots summarizing per-class band statistics (median, quartiles, outliers) for selected bands.
> `sample_data_quality.spectral_plotter.interactive_scatter_plot()`: produces an interactive 2D scatter plot of two bands with class coloring and hover details for exploring class clustering.
> `sample_data_quality.spectral_plotter.static_scatter_plot()`: renders a static 2D scatter plot with optional confidence ellipses
> `sample_data_quality.spectral_plotter.add_elipse()`: draws a confidence ellipse axis for a class' 2D band distribution (used by `static_scatter_plot`).

`sample_data_quality.spectral_plotter.scatter_plot_3d()`: builds an interactive 3D scatter plot to explore three-band feature space and rotate/zoom clusters.
`sample_data_quality.compute_tsne_coordinates()`: Compute 2D or 3D t-SNE embedding coordinates on standardized predictor features. Returns a DataFrame containing TSNE_1, TSNE_2 (, TSNE_3) and class metadata.
`sample_data_quality.compute_pca_coordinates()` : Compute PCA projection coordinates, explained variance ratios, and feature loadings on standardized predictor features.

5 Predictor Generation

Module Overview

The aim of Module 5 is to derive a set of predictors that optimize the discrimination of LULC classes in the classification model. Currently, Luma Geospatial Engine supports predictors derived from several categories, namely spectral transformations, topographical data, distance-based metrics, and radar backscatter. These features are critical because raw spectral data alone often fail to adequately separate spectral or spatially similar classes, and their inclusion enhances model accuracy and robustness.

The module outputs a comprehensive predictor stack that serves as input for LULC map generation in Module 6.

Input

Name of Input	Input Type	Details
spectral transformations	System's input	Retrieve the pre-define function from existing library
Topographical Data	System's input	
Distance-based metrics	System's input	
Radar Backscatter	System's Input	
AOI	Input from Other Modules	Output from Chapter 1
Near-cloud free satellite imagery	Input from Other Modules	Output from Chapter 1
Sample data	Input from Other Modules	Output from Chapter 3

Output

Predictor/covariate/feature stack used in the classification model (Module 6)

Process

5.1 Predictor Availability Check and Selection

1. The Luma GE supports various predictors that can be compute on the fly or manually uploaded by the user. The supported predictor categories are as follow:
    - Spectral transformation
    - Topographical data
    - Distance from feature
    - Radar Backscatter
2. The system perform confirmation availability of user selected predictors by confirming sensor selection and temporal coverage

> Related Function
>
> `predictor.predictor_catalog()` : central registry for supported predictor in Luma GE
> `predictor.predictor_catalog.get_all_catalog()` : Return a copy of the entire predictor catalog dictionary.
> `predictor.predictor_catalog.check_availability()` : Evaluate availability for all registered predictors based on sensor and temporal range.
> `predictor.sensorDetector()` : Utility for detecting satellite sensor category and resolving sensor-specific band mappings.
> `predictor.sensorDetector.normalize_sensor_name()`: Normalize a sensor string hint into a canonical sensor family.
> `predictor.sensorDetector.detect_sensor_from_bands()`: Detect sensor category from a list of band names. Sentinel-2 contains "Red-edge" bands, while Landsat contains "thermal" band, which can serves as sensor identifier
> `predictor.sensorDetector.detect_sensor()`: wrapper function to automatically detect sensor selection either using sensor name or band names

5.2 Predictor Computation

1. Topographical data

   - **Elevation**: Refers to Digital Elevation Model (DEM) products, widely used in land cover classification framework. Several DEM is supported by Luma-GE, namely NASADEM (default), Copernicus GLO-30, and ALOS DSM V4. All of the aforementioned DEM product have the spatial resolution of 30 m
   - **Slope**: Calculated from selected DEM using `ee.Terrain.slope` function. Slope unit is presented in degree units (0-90)
   - **Aspect**: Similar to slope, this data is calculated from the selected DEM, using `ee.Terrain.aspect`. The units are degree where 0=N, 90=E, 180=S, 270 =W
   - **Landform**: Refers to physical terrain attributes and classifications calculated from grid-based surface elevation values. Luma use global ALOS Landform dataset created by combining the Continuous Heat-Insolation Load Index (CHILI) and the multi-scale Topographic Position Index (mTPI) data

   > Related Functions
   >
   > `predictor.terrain_calculator.calculate_elevation` : Function to retrieve supported DEM data from GEE catalog.
   > `predictor.terrain_calculator.calculate_slope` : Core function to calculate slope from selected DEM data
   > `predictor.terrain_calculator.calculate_aspect` : Core function to calculate aspect data from selected DEM data
   > `predictor.terrain_calculator.get_landform` : Core function to retrieve global landform data and clip it based on AOI

2. Distance-based metrics

   The system calculates distance from several features that might affect the presence of certain land cover class. The selection and distance calculation of these features are inspired by RESTORE+ project. Distances can be measured from various infrastructures, primarily roads, coastlines, and settlements. Since distance calculation required heavy computation, pre calculated euclidean distance will be stored in the asset for three province in Indonesia, namely South Sumatera, South Sulawesi, and East Nusa Tenggara. For other regions, distance computation will be calculated on the fly currently supporting distance from primary roads. The road feature collection use as the basis for distance calculation, utilize Global Road Inventory Projects (Meijer et al. 2018).

Related Functions

`predictor.distance_calculator.calculate_distance_metrics`: Core function to initialize the distance calculation logic for coastline, road, and settlement
`predictor.get_predefine_distance` : function to retrive pre-calculated distance from main road raster from earth engine asset.

Currently, the system support distance calculation from the following source:

- Coastline: Natural Earth 10m coastline data,
- Settlement: Facebook High Resolution Settlement Layer (HRSL)
- Road: Indonesian Topographic Map (*Rupa Bumi Indonesia*) and OpenStreetMap

3. Spectral transformation indices

   Luma utilize open-source Awesome Spectral Indices (ASI) library, a standardized, ready-to-use curated list of spectral indices that can be used as expressions for computing spectral indices in remote sensing applications. Therefore, manually defining the expression is not required. However, when adding a new index, it should be cross-reference with the availability of the index in the library. For further reading on the formulation of the library and its supported indices, see (Montero et al. 2023). Currently, the supported index in Luma are as follows:

   - Vegetation-related indices:
     - NDVI (Normalized Difference Vegetation Index)
     - EVI (Enhanced Vegetation Index)
     - SAVI (Soil Adjusted Vegetation Index)
     - MSAVI (Modified Soil Adjusted Vegetation Index)
     - GNDVI (Green Normalized Difference Vegetation Index)
     - MCARI1 (Modified Chlorophyll Absorption in Reflectance Index 1)
   - Water-related indices:
     - MNDWI (Modified Normalized Difference Water Index)
     - ANDWI (Augmented Normalized Difference Water Index)
     - NDMI (Normalized Difference Moisture Index)
     - AWEInsh (Automatic Water Extraction Index with Shadow Elimination)
     - NDTI (Normalized Difference Turbidity Index)

- Soil / built-up-related indices:
  - NDBI (Normalized Difference Build-Up Index)
  - DBSI (Dry Bareness Soil Index)
  - MBI (Modified Build-up Index)
  - BLFEI (Built-Up Land Feature Extraction Index)

> Related Functions
>
> `predictor.SpectralCalculator.get_avaliable_indices`: Return the list of supported indices. This list contain information regarding the Index name, long-name, formula, and references
> `predictor.SpectralCalculator.get_index_coefficients`: Get default coefficient values for a spectral index that needs user-define coefficients and parameters
> `predictor.SpectralCalculator.validate_coefficients`: Conducts checks the provided coefficients are within reasonable ranges and provides warnings for unusual values that might indicate errors.
> `predictor.SpectralCalculator.validate_index_requirements` : Validate that required bands are available for index computation.
> `predictor.SpectralCalculator.calculate_indices_with_collection`: Conducts calculation for each scene (image collection) and then aggregate the result using mean reducers. This logic is used to preserve phenological pattern of vegetation and other land features which ultimately loss during if the final image composite is used.
> `predictor.SpectralCalculator.get_calculation_summary` : provide summary of the calculation process of predictors

4. Radar Backscatter

   Luma-GE supports Synthetic Aperture Radar (SAR) data as predictors for classification. Because radar observations are not affected by cloud cover, they can provide useful information in cloud-prone regions and complement optical imagery. The supported datasets are:

   - Sentinel-1 C-Band Dual Polarization

     Sentinel-1 carries a dual-polarization C-band SAR instrument. In Google Earth Engine, the available data are Ground Range Detected (GRD) products processed with the Sentinel-1 Toolbox to produce calibrated, orthorectified imagery. Luma-GE converts the log-scaled digital number (DN) values to normalized backscatter values. The dataset is available from 2014, has a six-day revisit interval at the equator, and provides 10 m spatial resolution.

- ALOS PALSAR/PALSAR-2

  The Advanced Land Observing Satellite (ALOS) carries the Phased Array type L-band Synthetic Aperture Radar (PALSAR). The PALSAR record begins in 2007 and has 25 m spatial resolution. A gap in coverage between ALOS and ALOS-2 means that data are unavailable from 2011 to 2014. Luma-GE currently supports yearly PALSAR mosaics, with the latest coverage available from January 2024. PALSAR ScanSAR data are not supported because they require extensive preprocessing.

The Sentinel-1 preprocessing workflow refers to (Mullissa et al. 2021) in the following repository: S1_ard. Pre-processing for Sentinel-1 include IW mode filtering, incidence-angle masking, low-backscatter swath-edge masking and speckle filtering (optional). Since PALSAR dataset already a yearly mosaic product, pre-processing only include radiometric calibration (conversion from DN to Db).

> **Related Function**
>
> `sar.get_sar_data:` Retrieve and preprocess SAR backscatter polarizations and polarization ratios as predictors for LULC mapping.
> `sar.get_sar_data.apply_speckle_filter():` function to perform speckle filtering for calibrated gamma-naught (dB) SAR image
> `sar.get_sar_data.mask_s1_border_and_angle():` Mask Sentinel-1 pixels with incidence angle outside [min_angle, max_angle] and low-backscatter swath-edge noise (VV or VH <= edge_db_floor).
> `sar.get_sar_data.get_sentinel1():` Build sentinel-1 backscatter predictor image
> `sar.get_sar_data.mask_and_calibrate_palsar():` Mask invalid PALSAR pixels using the 'qa' band and calibrate 16-bit DN to gamma-naught (dB).
> `sar.get_sar_data._resolve_palsar_year():` resolve the available PALSAR mosaic years intersection
> `sar.get_sar_data.get_palsar():` Build PALSAR backscatter predictor image. The function handle the following PALSAR collection: `JAXA/ALOS/PALSAR/YEARLY/SAR` (2007 - 2010) and `JAXA/ALOS/PALSAR/YEARLY/SAR_EPOCH` (2015 - 2024)

The list of radar backscatter band for each sensor are as follow:

1. Sentinel-1
   - VV: single co-polarization, vertical transmit/vertical receive
   - VH: single co-polarization, vertical transmit/horizontal receive
   - VV-VH Ratio: Ratio between VV and VH polarization
2. ALOS PALSAR

- HH: single polarization, horizontal transmit/horizontal receive
- HV: single polarization, horizontal transmit/vertical receive
- HH-HV Ratio: Ratio between HH and HV polarization

The main orchestration of predictor calculation is conduced after the system validate the required input. The entry point for SAR predictor is define through this source code The system combines the additional predictors chosen from this module with the multispectral bands from Module 1.

Related Functions

`predictor.PredictorCalculation.validate_prerequisites`: Validate that required inputs are available for predictor computation. Required input are composite (`ee.Image`), AOI (`ee.FeatureCollection`) , and raw image collection (`ee.Image.Collection`)
`predictor.PredictorCalculation.compute_predictors` : Initialize predictor computation by validating inputs, calculating selected terrain and spectral metrics, and stacking them into a final output.
`predictor.PredictorCalculation.prepare_module6_summary`: Prepare summary metadata for module 6
`predictor.PredictorCalculation.resolve_date_range()`: resolve date range selection between optical imagery date selection with radar date selection. Therefore, radar date range will always follow optical date selection
`predictor.PredictorCalculation.resolve_sar_selection()`: determine which radar sensor is selected based on user optical sensor selection

5.3 Predictors Visualization and Analysis

This section allow the user to visualize the resulted predictors to visually inspect the spatial distribution of the data. Predictor quality analysis is heavily related with sample quality analysis modules.

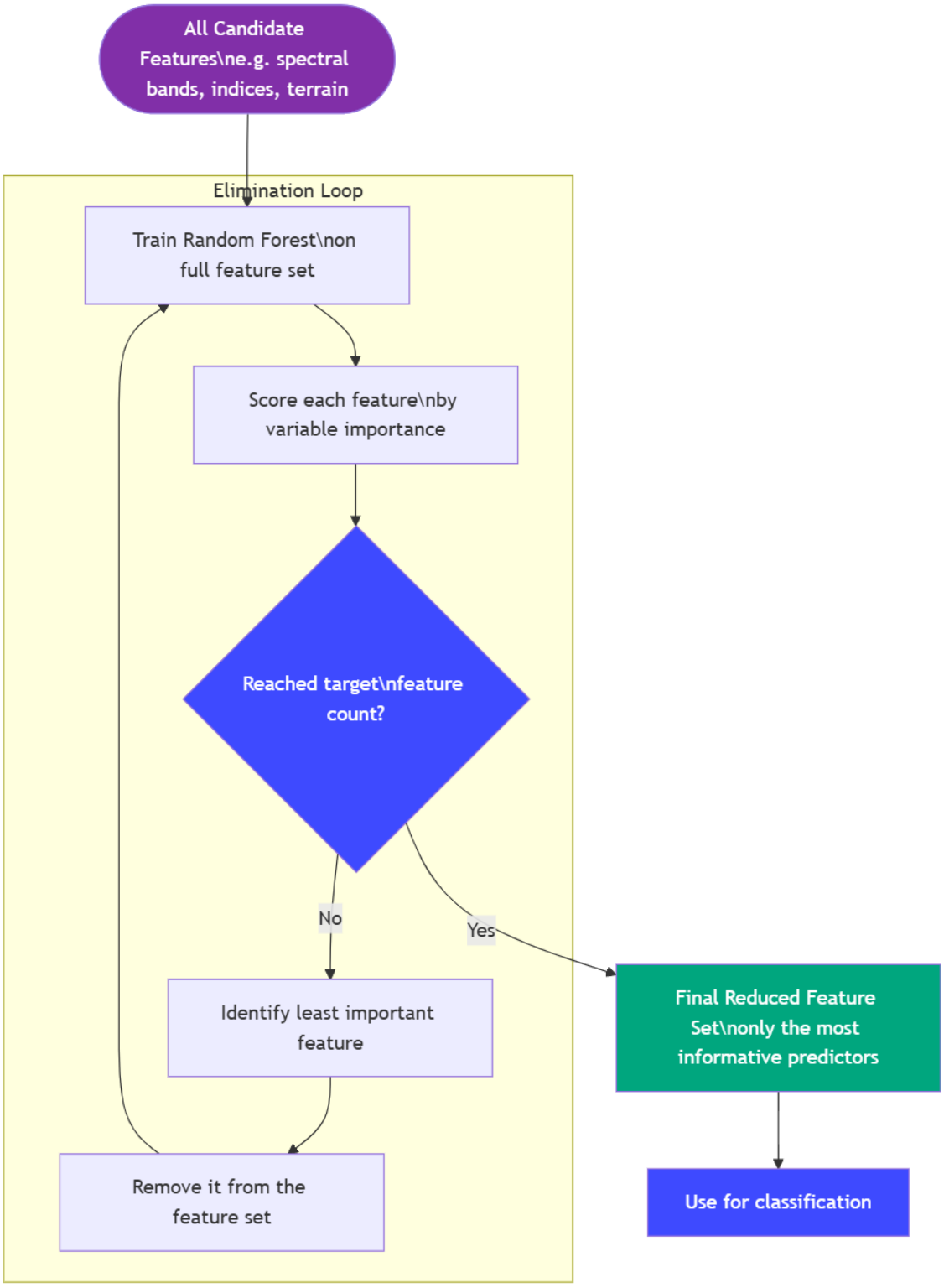
1. If the option to conduct correlation analysis was selected by the user, the system conduct correlation analysis using the following approach
    - Creates a random sampling across the AOI (default value is 5000 points)
    - Extract the pixel value of the computed predictors. The spectral band predictors are not extracted, since the analysis is design to analyze the correlation between ancillary predictors
    - Computes correlation analysis using GEE's core function

2. The correlation analysis is conducted across all predictors, including the original multi-spectral band

> Related Functions
>
> `predictor.EECorrelationAnalysis.generate_random_sampling`: Generate random sample across the AOI. Use default value of 5000 sample
> `predictor.EECorrelationAnalysis.compute_correlation`: Main function to compute Pearson correlation analysis based on the random samples generated. Return the panda data frame use for visualization.
> `predictor.EECorrelationAnalysis.visualize`: Visualize the correlation analysis result using heatmap

5.4 Semi Automatic Predictor Selection Optimization

Evaluating the user's predictor selection is crucial step in determining the best combination of predictors that resulted in the optimal model performance. This approach is more commonly known as feature engineering in machine learning terminology. One of the commonly use feature engineering approach is Recursive Feature Elimination (RFE). RFE applied a backward step-wise selection process to find optimal combination of predictors based on its contribution to the model. The RFE starts with training the model with all input predictors and discards the weakest predictor one iteration at a time, while re-evaluating their importance after each removal. The workflow for RFE is shown in the figure below:


All Candidate Features\ne.g. spectral bands, indices, terrain
Elimination Loop
Train Random Forest\non full feature set
Score each feature\nby variable importance
Reached target\nfeature count?
No
Yes
Identify least important feature
Remove it from the feature set
Final Reduced Feature Set\nonly the most informative predictors
Use for classification

1. Input validation

   Prior to running the RFE, Luma-GE is design to perform input validation checks just in case a missing parameter is detected. If the input is invalid, provide informative error messages. The invalid input includes missing data (training or testing) or missing ancillary predictors. If the input is valid, proceed with RFE calculation

   > Related Function
   >
   > `predictor.RFEGEE.validate_inputs()`:validate all input require for RFE calculations, namely: training data (`FeatureCollection`), test data (`FeatureCollection`), number of feature selected (>1), and predictors (`ee.Image`)

2. Default RFE configuration

   - Minimum feature to select: 10
   - Minimum random forest tree: 100
   - Training data (for RFE): 70% of user input
   - Test data (for RFE): 30% of user input

   Since Earth Engine did not have default feature selection algorithm, the Luma GE create looping mechanism that train random forest classifier and return feature importance ranking and accuracy of the model for each iterations. The code is inspired by similar approach by this github repository

> Related Function
>
> `predictor.RFEGEE._train_classifier()` : core function to perform random forest classification with default configuration.
> `predictor.RFEGEE._get_importance_and_accuracy()` :Retrieve feature importance test accuracy in a single round-trip to the Earth Engine servers. Optimizing the time required for finish the entire RFE.
> `predictor.RRGEE._adaptive_step()` : fallback mechanism to reduce API calls in case computational loads become larger.
> `predictor.RFGEE.fit()`: function to conduct the complete workflow for the entire RFE, including train classifier and calling model importance and its accuracy.

To optimize API calls in Earth Engine servers, the calculations is conducted using server-side computations, and only called after all iterations is complete. The result of RFE is visualize using graphic and tables, including the selected features, rank of eliminations, and accuracy curve. Currently, the user require to determine the number of selected predictors for RFE. Future development will optimize this features

Related Function

`predictor.RFEGEE.get_selected_features` Function to retrieve optimal predictors list based on RFE result

`predictor.RFEGEE.get_ranking` Function to rank importance of each predictors

`predictor.RFEGEE.plot_accuracy_curve` Plot the accuracy curve as each features is remove from original input features

`predictor.RFEGEE.plot_feature_rankings` Plot the feature/predictor importance from RFE calculations

`predictor.RFGEE.plot_rfe_summary` Create summary plot of accuracy curve and feature rankings

`predictor.RFEGEE.get_results_dataframe` Generate a dataframe of the corresponding accuracy and importance data

6 LULC Map Generation

Module Overview

This module performs supervised Random Forest classification to generate a land cover map. It includes input review, hyperparameter definition, feature extraction, model training, hard/soft classification, feature importance, and accuracy evaluation using test data.

Input

Name of input	Input type	Details
Sample data split proportion	User's input	Optional, if not set then use default value
Random Forest (RF) parameters	User's input	Optional, if not set then use default value
Satellite imagery within the AOI	Input from other modules	Module 1
LULC classes	Input from other modules	Module 2, including the selection of the classes of interest if the user uses the default classification scheme.
Sample data	Input from other modules	Module 3

Output

1. Trained LULC classification model.
2. Model accuracy report if validation data available (including overall accuracy, precision, recall, F1-score, Kappa, model error matrix). This output is only available if the user has a validation data from the splitted the sample data
3. Classified LULC raster map.

Process

6.1 Extracting spectral feature from sample data

6.1.1 Splitting the sample data

1. The system conducts feature extraction for both training and validation data accordingly to the proportion set by the user.
2. The system uses stratified random split method to conduct the data splitting. This method randomly selects samples within each class to proportionally create training and testing sets while preserving class distribution.

> Related Functions
>
> `classification.FeatureExtraction.stratified_split()` : performs stratified random split of sample data based on LULC class labels.

6.2 Creating classification model

6.2.1 Creating classification model

1. The system create the classification model using hard classification method.

> Related Functions
>
> `classification.Generate_LULC.hard_classification()` : trains a RF classifier (`smileRandomForest`) using extracted training data and applies it to the input imagery to produce a hard (pixel-level) multiclass.

6.2.2 Reviewing classification model's quality

1. The system extracts feature importance from the trained Random Forest model and presents a ranked table of bands (or a fallback equal-weight estimate if true importance is unavailable).

   > Related Functions
   >
   > `classification.Generate_LULC.get_feature_importance()` : extracts and returns model feature importance as a dataframe; if the model explanation is not available, falls back to an equal-weight estimate.

2. The system calculates model evaluation metrics on the validation dataset. This option only available if the user perform data split.

   > Related Functions
   >
   > `classification.Generate_LULC.evaluate_model()` :classifies the validation data with the trained classifier, builds the confusion matrix, and returns a dictionary of accuracy metrics

3. Confusion matrix and model accuracy summary are then rendered in the user-interface.

6.2.3 Visualizing LULC classification map result

1. The system loads the user's class of interest stored from Module 2.
2. The system reclassifies the default classification scheme's map into a map that only shows the user's class of interest and an additional class labeled as "Others".

   > Related Functions
   >
   > `classification.Generate_LULC.reclassify_map_by_classes()`: reclassifies the default classification scheme final map to only show the user's class of interest and show the other classes as "Others".

6.2.4 Downloading the LULC classification map result

1. This sub-step does not involve any operations from Luma Geospatial Engine. Direct download settings are defined from Streamlit, using the same workflow as defined in Section 1.5.

2. For details on exporting to Google Cloud Storage, refer to Earth Engine Initialization and Authentication page.

6.3 Default classification scheme classification from probability layers

When the user chose to use default classification scheme, Section 6.1 and Section 6.2 are skipped.

1. The system loads the pre-generated probability map stack and clips it to the user's AOI.

   > Related Functions
   >
   > A function to load & prepare the pre-generated probability map

2. If the user selects only a subset of classes from the classification scheme, the system retrieves the selected classes and discards the probability maps for the unselected classes.

   > Related Functions
   >
   > A function to load the chosen classification scheme

3. For each pixel, the system retrieves the probability value from each layer and assigns a class based on the highest probability value using the argmax function.

   > Related Functions
   >
   > A function to apply argmax

4. Post-classification cleanup is performed by masking the classified map according to a predefined set of rule-based predictors.

   > Related Functions
   >
   > A function to conduct post-classification

7 Thematic Accuracy Assessment

Module Overview

This module performs a comprehensive thematic accuracy assessment of the land cover map using an independent ground reference data. The framework for this module are based on the publication, *Good Practices for Estimating Area and Assessing Accuracy of Land Change*, by Olofsson et al 2014. This framework separates accuracy assessment into three major components, namely sampling design, response design, and analysis. The Luma-GE only adapt the sampling design and analysis components. Since the response design consist of protocols and guideline in obtaining the ground reference data, this component are currently not implemented.

Input

Name of input	Input type	Details
Validation map	User's input	In shapefile format.
Classified LULC map	Input from Other Modules	Module 6

Output

- Confusion matrix.
- Thematic accuracy metrics: Overall Accuracy (OA), Kappa Coefficient, Producer's Accuracy (PA), User's Accuracy (UA).
- Reference data sites (optional, for users who do not have reference data, the system will generate reference data sites for them to label outside Luma-GE).
- Spatial distribution of error

Process

7.1 Checking Prerequisites from Previous Modules

The system validates availability of the required inputs.

7.2 Thematic Accuracy Assessment

7.2.1 Selecting The Workflow

Two workflow are available for Module 7. The first workflow is design for the user who did not have reference data. The second workflow is for the user who already have a reference data.

For each workflow, the geospatial engine perform the following operation

7.2.2 Generate Reference Sample Workflow

This workflow consist of two steps, steps, namely sample size calculation and sample allocation. The result of each steps is how many minimum sample required for each class and the location for the samples.

Sample Size Calculation

Sample size calculation for stratified random sample is calculated using the formula provided by Cochran (1977, Eq. (5.25)). The samples are proportionally allocated for each strata (class), resulting in balance sample allocation. The key steps for sample size calculation is as follows:

1. Calculate stratum standard deviation: `Si = √(Ui × (1 - Ui))`

   Where:

   Si = standard deviation for stratum i

   Ui = expected accuracy for class

2. Calculate stratum weight: `Wi = Ni / N`

   Wi = weight of stratum i

   Ni = Number of pixels in class i

   N = Total number of pixels

3. Calculate total sample size: `n = (Σ(Wi × Si) / SE)²`

   n = Total required samples

   SE = desire standard error

4. Allocate sample per stratum `ni = n × (Wi × Si) / Σ(Wi × Si)`

   ni = Number of samples for stratum i

> Related Function
>
> `sample_size_calculator.get_pixel_count_per_class:` calculate pixel count for each class using earth engine's reducer
> `sample_size_calculator.validate_sample_size_inputs:` Input validation prior to main sample size calculation
> `sample_size_calculator.calculate_strata_sample:` Main function that calculate sample size for each class

Sample Allocation

The required sample for each class is allocated in the area of interest using `ee.Image.stratifiedSample()` . The sample size are based on the sample size calculation result. The feature collection from this process only contain class ID for each land cover class. Class name will be added in the future update. The user are able to download the samples, therefore they can labeled the sample outside Luma GE. Current function only support shapefiles.

> Related Function
>
> `sample_size_calculator.generate_stratified_samples:` Generate the reference samples based sample size calculation
> `sample_size_calculator.export_samples_to_shp:` Export the generated samples

7.2.3 Accuracy Computation Workflow

1. The system verifies if the `.shp` of the validation map is the uploaded inside the `.zip` file.

> **Error Handling Notification**
>
> The system provides an error notification if the `.shp` file is not found inside the `.zip` file.

2. The system conducts geometry fixes on the uploaded validation map.

> **Related Function**
>
> `input_utils.shapefile_validator.validate_and_fix_geometry()`:to validate and fix geometries issues.

> **Error Handling Notification**
>
> An error message is displayed if the system failed to run `validate_and_fix_geometry()`.

3. The system converts the validation map from `.gdf` data into Earth Engine Feature Collection format

> **Related Function**
>
> `input_utils.shapefile_validator.convert_roi_gdf()`: to convert geo-dataframe into EE Feture Collection. The supported multi-geometries type for this conversions are MultiPoint and MultiPolygon.

> **Error Handling Notification**
>
> An error message is displayed if the system failed to run `convert_roi_gdf()`.

7.2.4 Starting the thematic accuracy assessment

1. The system performs a thematic accuracy assessment by validating inputs, sampling the classified LULC map at reference points.

> **Related Functions**
>
> `accuracy.thematic_accuracy.validate_assessment_inputs()`: checks whether the classified LULC map, validation map, LULC ID class field, and pixel size parameter meet the required conditions before the assessment runs. This function is used in `run_accuracy_assessment()`.

`accuracy.thematic_accuracy._extract_confusion_matrix_data()`: extracts overall accuracy, kappa, per-class accuracies, and the confusion matrix array from an `ee.ConfusionMatrix` object. This function is used in `run_accuracy_assessment()`.
`accuracy.thematic_accuracy._calculate_confidence_interval()`: Computes the confidence interval for overall accuracy using a normal approximation based on the number of correct and total samples. This function is used in `run_accuracy_assessment()`.
`accuracy.thematic_accuracy._calculate_f1_scores()`: Calculates the F1 score for each class using the producer's and user's accuracy values. This function is used in `run_accuracy_assessment()`.
`accuracy.thematic_accuracy.run_accuracy_assessment()`: Executes the full thematic accuracy workflow, including validation, sampling (`sampleRegions()`), metric extraction (`errorMatrix()`), confidence interval calculation, and compilation of final results.

2. Thematic Accuracy Metrics

   Several accuracy metrics is calculated to provide comprehensive report of the map's thematic quality.

   - **Overall Accuracy**

     Overall Accuracy (OA): Sum of the major diagonal (correctly classified pixels) divided by the total pixels in the entire confusion matrix

     `OA = Σ(n_ii) / N`

   - **Kappa Coefficient**

     Kappa coefficient is statistical test generated from the error matrix. Kappa coefficient show how well the classification performed as compared to just randomly assigning values. Kappa value range from -1 to 1, in which the value of 0 indicate that the classification is no better than a random classification. The negative value indicate that the classification is worse than random classification. The value closer to 1 indicate that the classification is better than random classification.

     `= (Po - Pe) / (1 - Pe) Po = Σ(n_ii) / N Pe = Σ(n_i+ × n_+i) / N²`

     Where:

     Po = Observed Aggreement (OA)

     Pe = Expected agreement by chance

- **Producers's Accuracy (Recall/Sensitivity)**

  One of the class level accuracy metric, which answer the question, *"What fraction of actual class was correctly mapped?"*

  `PA_i = n_ii / n_i+`

  Where:

  `n_ii` = correctly classified samples for class i `n_i+` = total reference samples for class i (row sum)

- **User's Accuracy (Precision)**

  One of the class level error metric, which answer the question *"What fraction of predicted class i is actually class i?"*

  `UA_i = n_ii / n_+i`

  Where:

  `n_ii` = correctly classified samples for class i `n_+i` = total predicted samples for class i (column sum)

7.2.5 Spatial Distribution of Error

This function provide reference data flagging for visualizing the spatial error distribution, improving the error analysis by tagging each point with whether the classifier got it right or wrong. Currently, this function only works if the reference data unit is point (single pixel). If the reference data consist of polygons, this function will not work.

1. The system conduct checks if reference data geodataframe is available.
2. Add actual and predicted class to the geodataframe as key point in determining correctness of the map
3. Generate a pop-up html with comprehensive information regarding the status of the corresponding reference sites

> Related Function
>
> `accuracy.validation_error_flag.classify_validation_points()`: core function to perform correctness flagging to the classification map
> `accuracy.validation_error_flag.generate_popup_html()`: create a pop up html for visualizing the error flag

7.2.6 Reviewing the thematic accuracy result

This sub-step does not involve any operations from Luma Geospatial Engine.

8 Post Classification Analysis

Under development

This module's backend is not available yet.

Part III

Luma Interface

1 Map Generation

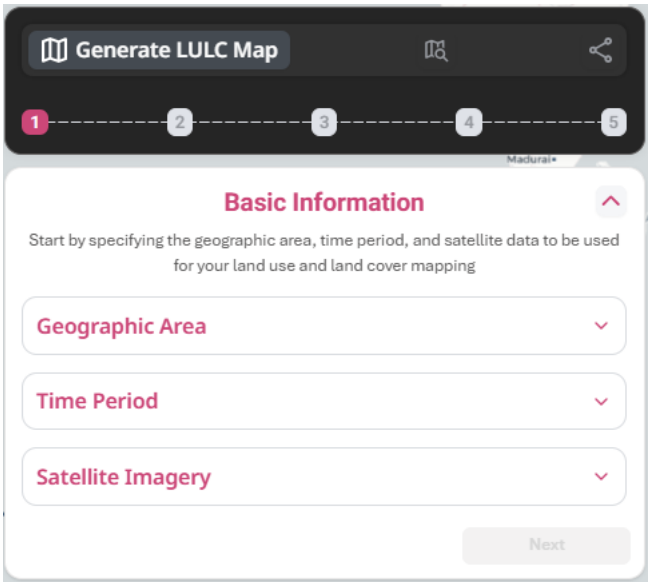
Overview

In this workflow, the user produces a Land Use and Land Cover (LULC) map. Each step is presented as a panel on the left of the map canvas and progresses via the indicator at the top of the panel. To ensure usability regardless of technical know-how, user cannot skip steps; each step unlocks the next only after the required input has been provided by the user and the Geospatial Engine (GE) confirms the corresponding output has been produced and saved.

Step 1: Basic Information

Overview

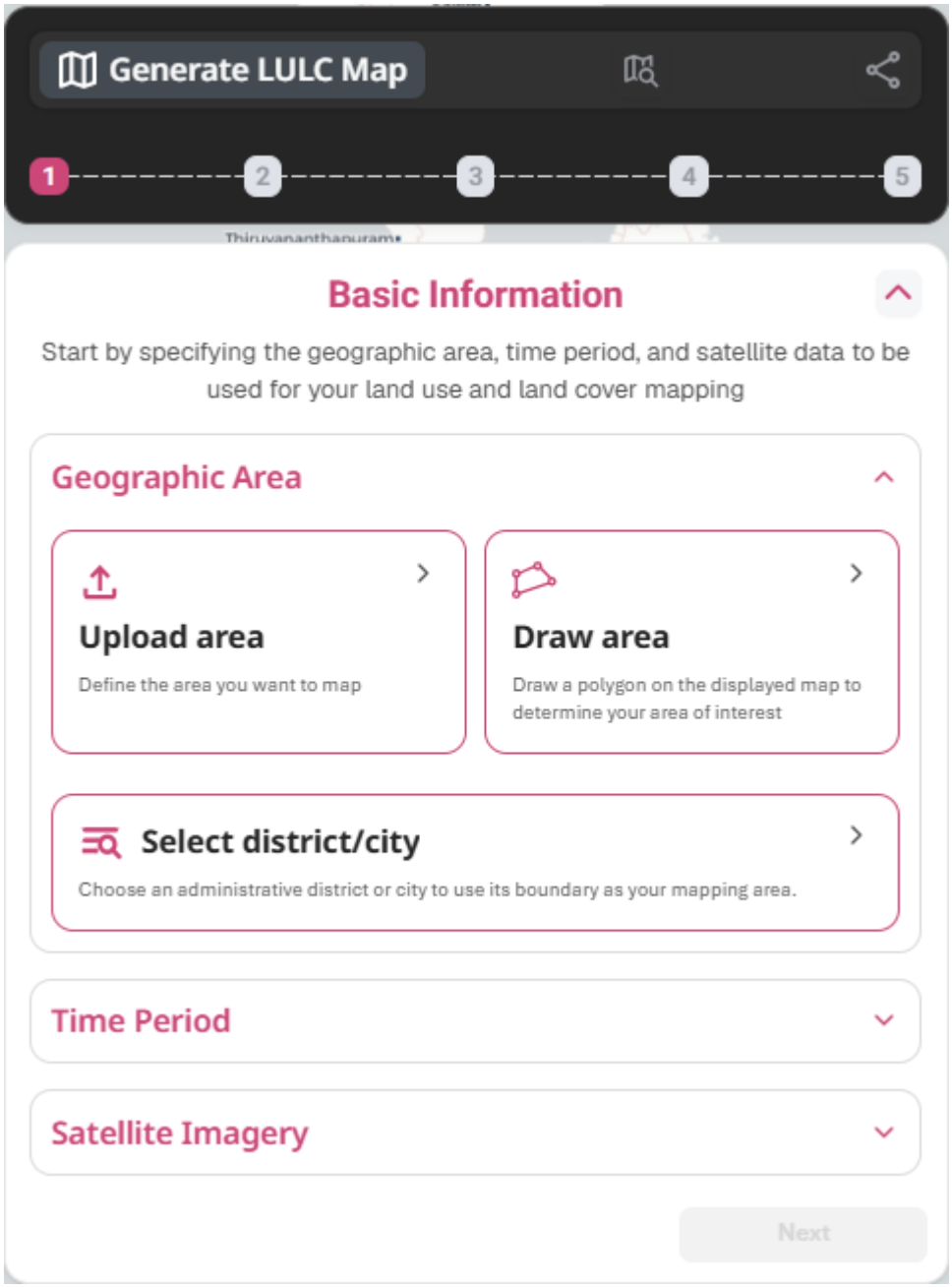
Provides the inputs needed before any geospatial computation: where to map, when, and from which satellite imagery source.



Step 1.1: Geographic Area

Overview

This is the first required input in the Map Generation workflow where the user specify the geographic area they want to map by choosing one of three methods available: upload, draw area, or select an administrative boundary.

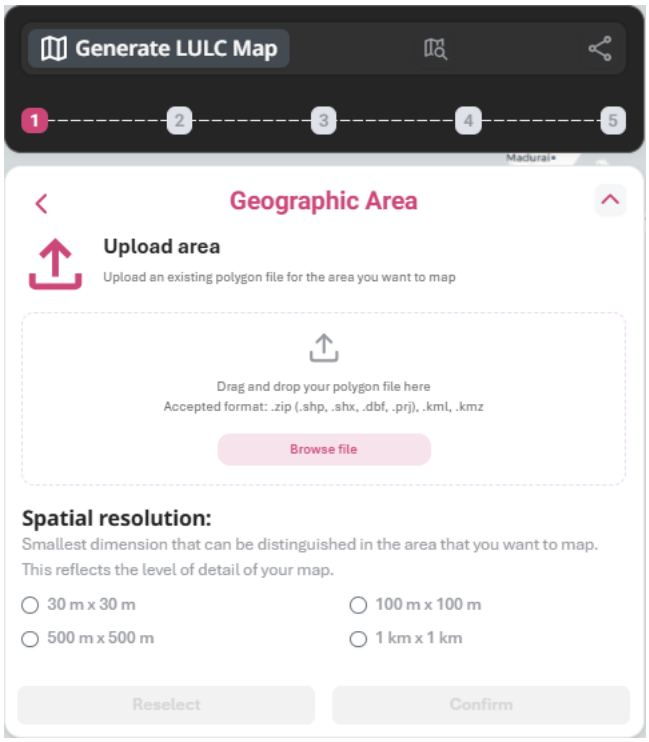


Step 1.1.a: Upload Area

Overview

The user uploads a polygon file to define the geographic boundary of the area they want to

map.



User Input to Interface

1. The user uploads their own AOI file in `.zip` format that should include all required shapefile components (`.shp`, `.shx`, `.dbf`, `.prj`). The system also supports uploads of `.kml` or `.kmz` files.
2. The user chooses a spatial resolution from the selections (10 m x 10 m, 30 m x 30 m, 100 m x 100 m, 500 m x 500 m, or 1 km x 1 km).

Process

1. The user can upload their own AOI file in `.zip` format that includes all required shapefile components (`.shp`, `.shx`, `.dbf`, `.prj`).
2. The system checks on all required shapefile components that needs to be put in the `.zip` file. The system also supports uploads of `.kml` or `.kmz` files.

> **Success Notification**
>
> If all shapefile components pass validation, continue to Process 3

> **!** Error Handling Notification
>
> If the shapefile components do not match, an error warning is displayed: 'No `.shp` file found in the `.zip` file.'

3. The system retrieves AOI file information.

> **i** Success Notification
>
> If all file information is correct, continue to Process 4

> **!** Error Handling Notification
>
> If the file information cannot be found, an error warning is displayed: 'AOI not found'

4. The system retrieves AOI file area size information.

> **i** Success Notification
>
> If the area size information is within the maximum area allowed, continue to Process 5

> **!** Error Handling Notification
>
> If the area size information is too large, an error warning is displayed: 'Draw area exceeds maximum limit'

5. The system validates the geometry of the AOI for both `.shp` and `.kml` using the `shapefile_validator` functions.

> **i** Success Notification
>
> If GIS file completeness is validated, continue to Process 6

> **!** Error Handling Notification
>
> If validation failed, an error warning is displayed: 'Geometry validation failed.'

6. The system returns the validated and processed shapefile.

> **Success Notification**
>
> The system shows a confirmation when the AOI is loaded successfully by showing a purple box with 'Approximate size of your mapping area: X [number] Ha' displayed

7. The system activates the next step.
8. The user chooses a spatial resolution selection (10 m x 10 m, 30 m x 30 m, 100 m x 100 m, 500 m x 500 m, or 1 km x 1 km).
9. The system returns the validated and processed shapefile.

> **Success Notification**
>
> The "Confirm" button is activated

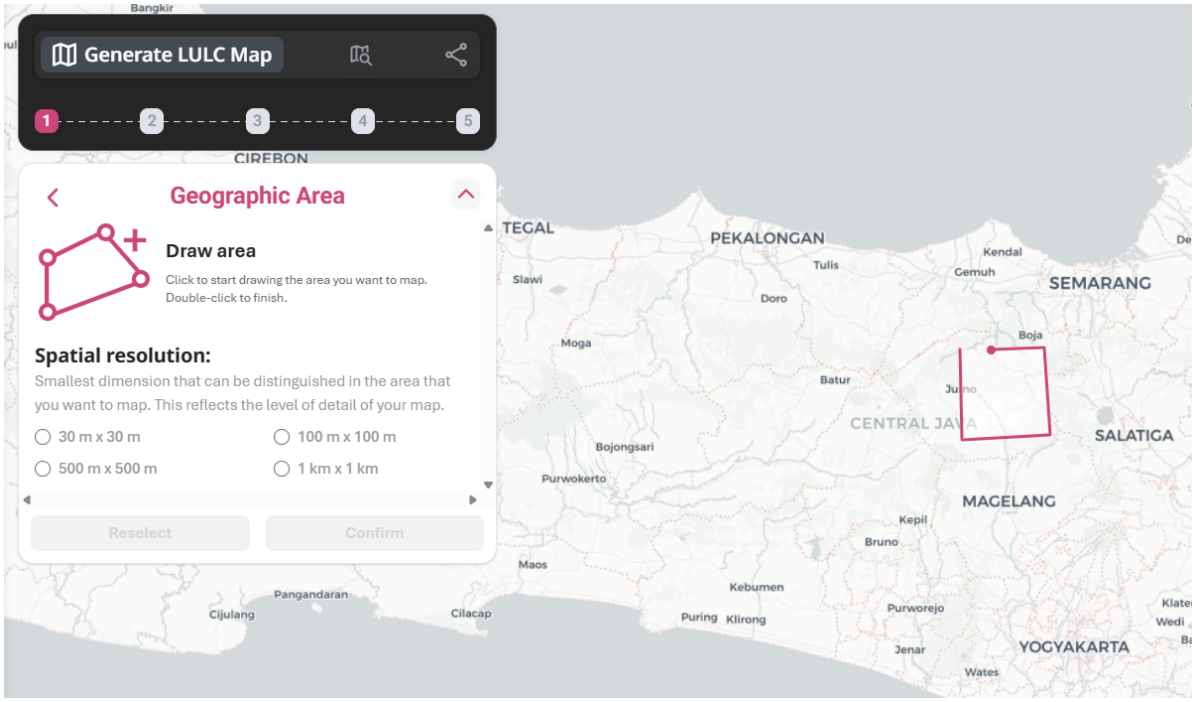
10. If the user clicks "Reselect Area", the uploaded file is deleted.

> **Success Notification**
>
> All inputs are being reset, the user will be redirected to **?@sec-map-generation-basic-information**

Step 1.1.b: Draw Area

Overview

The user draws their area of interest directly on the map by creating borders of area they want to map. Each click adds a new side/edge (vertex) to the polygon, with a double-click to finish drawing and close the polygon. This is useful when the user does not have a prepared polygon file or wants to quickly define a simple area.



User Input to Interface

1. The user creates a series of clicks on the map to create a polygon.
2. The user chooses a spatial resolution from the selections (10 m x 10 m, 30 m x 30 m, 100 m x 100 m, 500 m x 500 m, or 1 km x 1 km).

Process

1. The user creates a polygon by clicking on the map.
2. The system retrieves the AOI area size information.

> **Success Notification**
>
> The system shows a confirmation when the AOI is loaded successfully by showing a purple box with ‘Approximate size of your mapping area: X [number] Ha’ displayed

> **Error Handling Notification**
>
> If the area size information is too large, an error warning is displayed: ’Draw area exceeds maximum limit’

3. The system activates the next step.

4. The user chooses a spatial resolution selection (10 m x 10 m, 30 m x 30 m, 100 m x 100 m, 500 m x 500 m, or 1 km x 1 km).
5. The system returns the validated and processed shapefile.

   > **Success Notification**
   >
   > The "Confirm" button is activated

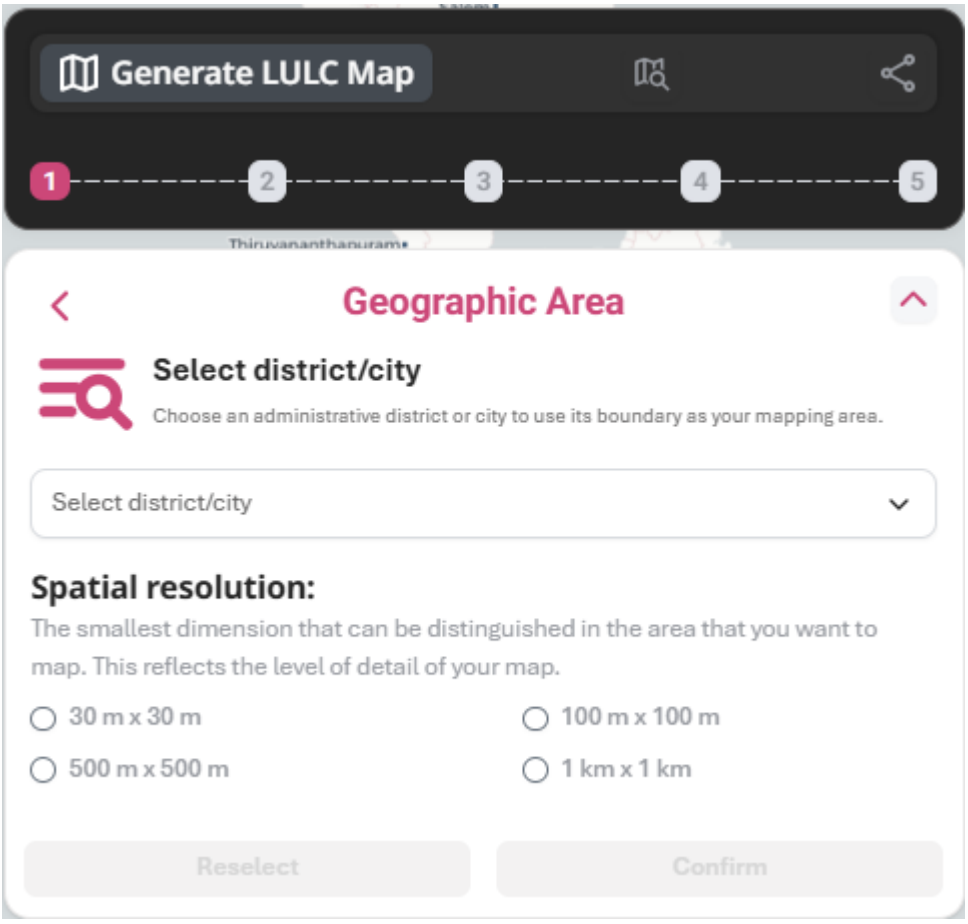
6. If the user clicks "Reselect Area", the drawn polygon is deleted, and the user will be redirected to **?@sec-map-generation-basic-information**.

   > **Success Notification**
   >
   > All inputs are being reset, the user will be redirected to **?@sec-map-generation-basic-information**

Step 1.1.c: Administrative Boundaries

Overview

The user selects the area they want to map based on a drop down list of Indonesian regency administrative boundaries. Disclaimer about the data source (Badan Informasi Geospasial) is included with the data uploaded as GEE asset and the link provided and shared publicly. The system provides a preview of the selected boundary.



User Input to Interface

1. The user selects an AOI based on a drop down list of Indonesian regency administrative boundaries.
2. The user chooses a spatial resolution from the selections (10 m x 10 m, 30 m x 30 m, 100 m x 100 m, 500 m x 500 m, or 1 km x 1 km).

Process

1. The user selects an administrative boundary from the drop down list.
2. The system retrieves the AOI information from the selected administrative boundary selections.
3. The system provides the name of selected administrative boundary.
4. The system activates the next step.

5. The user chooses a spatial resolution selection (10 m x 10 m, 30 m x 30 m, 100 m x 100 m, 500 m x 500 m, or 1 km x 1 km).
6. The system runs the related functions in Section 1.1.2 and provides a polygon of the selected boundary on the map canvas.

   > **Success Notification**
   >
   > The "Confirm" button is activated

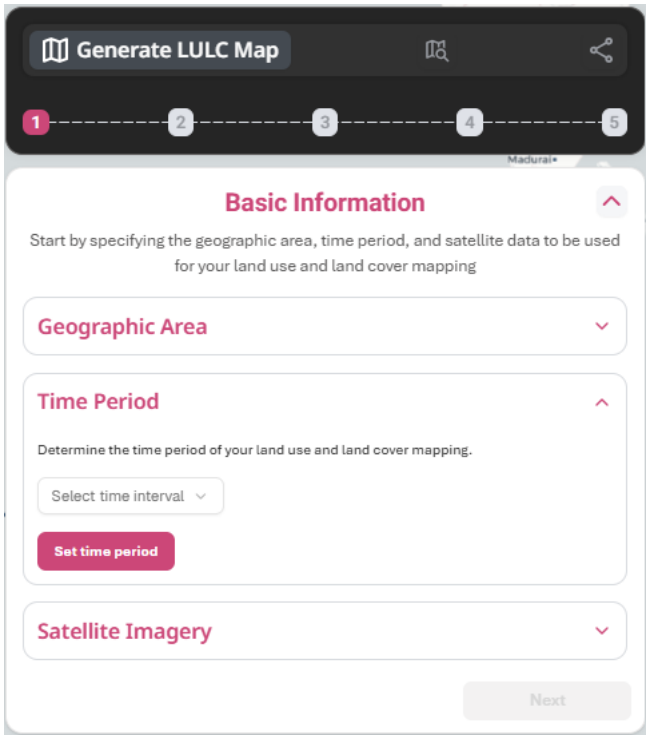
7. If the user clicks "Reselect Area", the selected administrative polygon is deleted, and the user will be redirected to **?@sec-map-generation-basic-information**.

   > **Success Notification**
   >
   > All inputs are being reset, the user will be redirected to **?@sec-map-generation-basic-information**

Step 1.2: Time Period

Overview

The user defines the time period for the land use and land cover map. Currently available options for time interval of the satellite imagery period are limited to yearly (a single year) or custom date range.



User Input to Interface

1. The user selects the time interval they want to use (yearly or custom date range).
2. The user further specifies the period by selecting the year (if yearly) or entering the start and end dates (if custom date range) that they want to map.

Process

1. The user chooses a time interval selection “By year” or ”Custom date range”.
2. The system provides input fields to specify time period based on selected time interval.
3. If “By year”: The system shows the year selections from 1972 until current year in a form of a drop down list.
4. If “Custom date range”: The system provides calendar display to allow user choose specific start and end dates.
5. The user selects a year selection (1972–current year) or specific start and end dates.
6. The system activates “Set time period” button.
7. If the user clicks “Set time period” button, the system confirms the time period input.

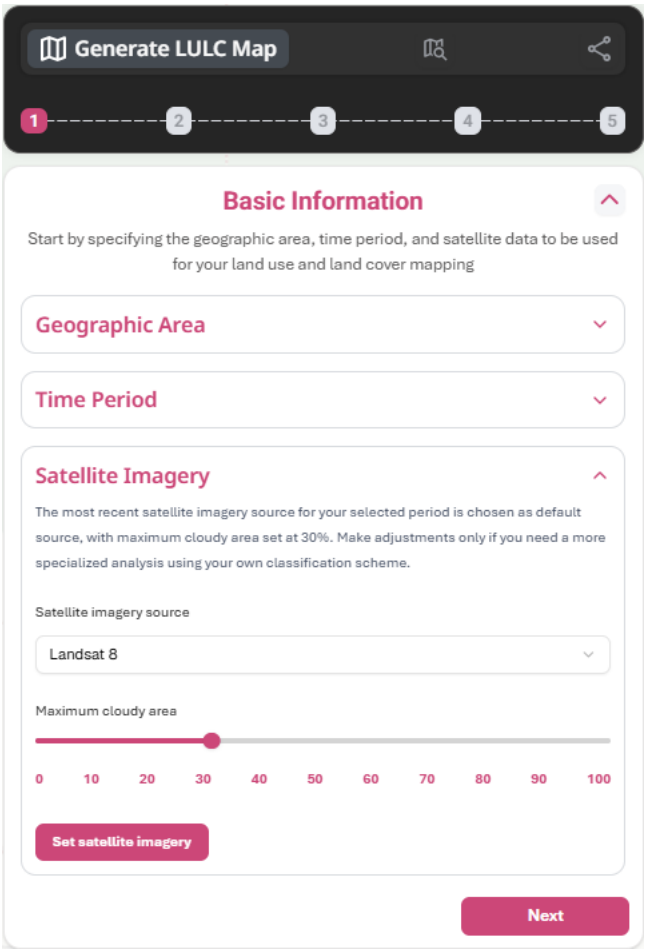
> **Success Notification**
>
> - The system shows a confirmation by showing a purple box with a summary of: time interval, period, satellite imagery date range selected
> - A clickable "Change input" text also appears

8. If the user clicks "Change input" button, the system resets all input from **?@sec-map-generation-time-period** and is brought back to the beginning of **?@sec-map-generation-time-period**.

Step 1.3: Satellite Imagery

Overview

The user confirms default selection for satellite imagery source and the maximum cloud coverage percentage or click change input to customize final input to the basic information.



User Input to Interface

1. Most recent available satellite imagery source and 30% maximum cloudy area are chosen as default input. User is provided with the option to "Change input". If chosen, user can input 2 and 3 below.
2. The user selects the satellite imagery source from a drop down list.
3. The user selects the maximum cloudy area by dragging the slider to the desired number.

Process

1. The system adjusts default satellite imagery source and other options according to time period input in **?@sec-map-generation-time-period**.
2. The user sets a preferred cloud cover threshold, if none chosen, the default value of 30% is used.

3. The system performs pre-processing functions in Section 1.3.1.

> **Success Notification**
>
> When all information is collected, continue to Process 4

> **Error Handling Notification**
>
> The system provides an error notification if no satellite images were found based on the user's parameter. The user is prompted to change the cloud cover threshold or change the time period.

Step 1.4: Confirm and Preview

Overview

After all input to basic information have been finalized, user is presented with summary of input for final confirmation. User confirms through clicking "View satellite imagery" which will trigger the system to retrieve all available scenes, removes cloudy pixels, and combines the rest into a single cloud-free composite image. This composite is the base image for all classification steps. Three visualisation modes are generated: True Colour (RGB), False Colour Infrared, and Land/Water.



User Input to Interface

1. The user clicks Next to confirm summary of all inputs after **?@sec-map-generation-satellite-imagery**.

2. The user clicks View satellite imagery to initiate processing of satellite imagery.

Process

1. The system summarizes the basic information from previous steps.

   > **Success Notification**
   >
   > The user's input summary is displayed in a purple box, showing:
   >
   > - Mapping area: approximate size of your mapping area
   > - Time period selected: time interval, selected period, satellite input date range
   > - Satellite imagery: satellite imagery source, maximum cloudy area
   >
   > Two buttons appear:
   >
   > - "Change Input"; continue to Process 2
   > - "View Satellite Imagery"; continue to Process 5

   > **Error Handling Notification**
   >
   > The system provides an error notification if no satellite images were found based on the user's parameter. The user is prompted to change the cloud cover threshold or change the time period.

2. If the user clicks on the "Change Input" button, "Edit" (pencil icon) buttons appear next to each of the input summary's heading. Once clicked, a pop up window appeared for user to choose:

   - "Change Input", continue to Process 3, or
   - "Discard Changes", continue to Process 4.

3. If the user clicks on the "Change Input" button in the pop up window, all inputs are being reset, back to the beginning of **?@sec-map-generation-basic-information**.

4. If the user clicks on the "Discard Changes" button in the pop up window, the panel still displays the user's input summary. "Cancel Change" button appear, while "View Satellite Imagery" is disabled. Once the "Cancel Change" button is clicked, back to Process 1.

5. If the user clicks on the "View Satellite Imagery" button, the system processes the satellite image request.

Success Notification

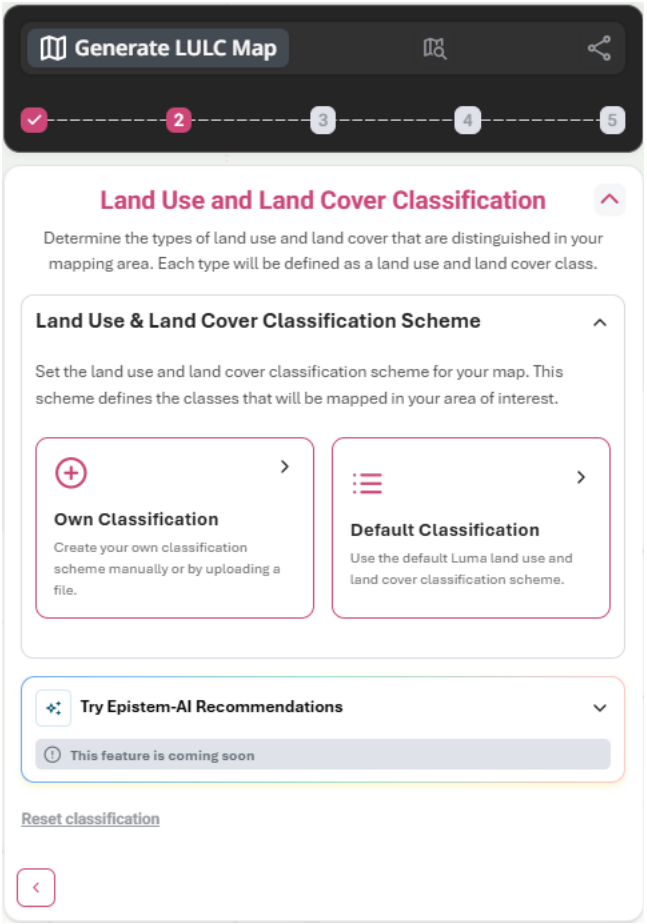
- A table of Generated Composite of Imagery Obtained appeared under the User's Input Summary along with Cloud Coverage Statistic.
- Composite of satellite imagery appears on the base map, with default setting of true colour (RGB).
- Layers option appears on the bottom right corner of the screen, includes:
  - Mapping area; with polygon toggle
  - Composite of satellite imagery; with true colour (RGB), false colour infrared (NIR/Red/Green), land/water (NIR/SWIR1/RED) toggle options
  - LULC class; still disabled in this step
- User can then download the composite, leading to Process 6.
- "Next" button appears, which leads to **?@sec-map-generation-lulc-classification**.

6. If the user downloads composite, a `.zip` file containing satellite image composite is downloaded to user's local device.

Step 2: Land Use and Land Cover Classification

Overview

The user defines the land use and land cover classes that will be mapped. There are two main options: using the built-in Luma default classification scheme or uploading a custom classification scheme. The selected classes determine which training data will be used in the next step. The Epistem-AI feature will also be available here.



Step 2.1: Land Use & Land Cover Classification Scheme

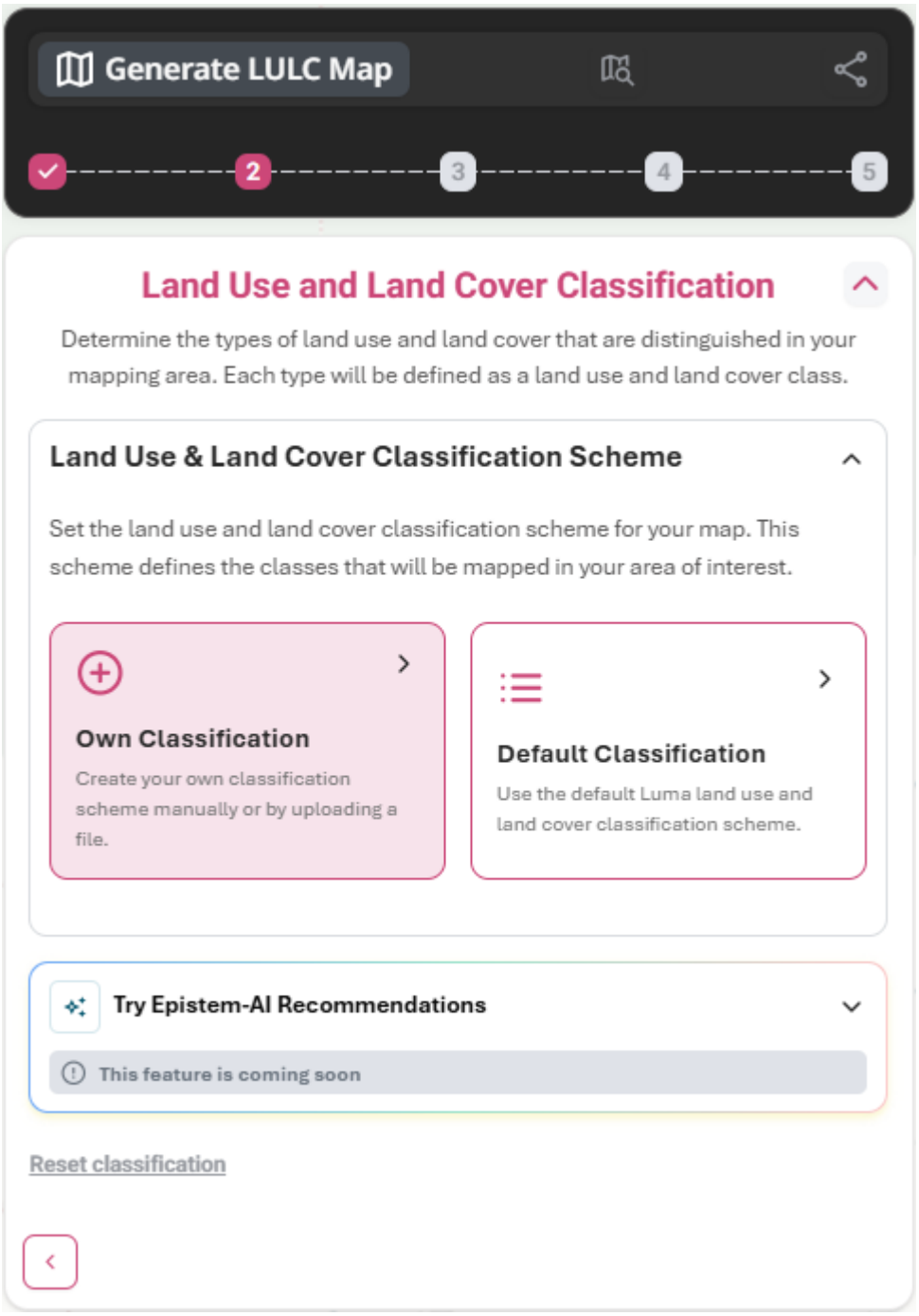
Overview

Luma supports user-defined classification schemes and default classification or templates for commonly used classification schemes.

Step 2.1.a: Own Classification

Overview

The user creates their own classification scheme manually or by uploading a file.



User Input to Interface

1. The user clicks Own Classification to create their own classification scheme.

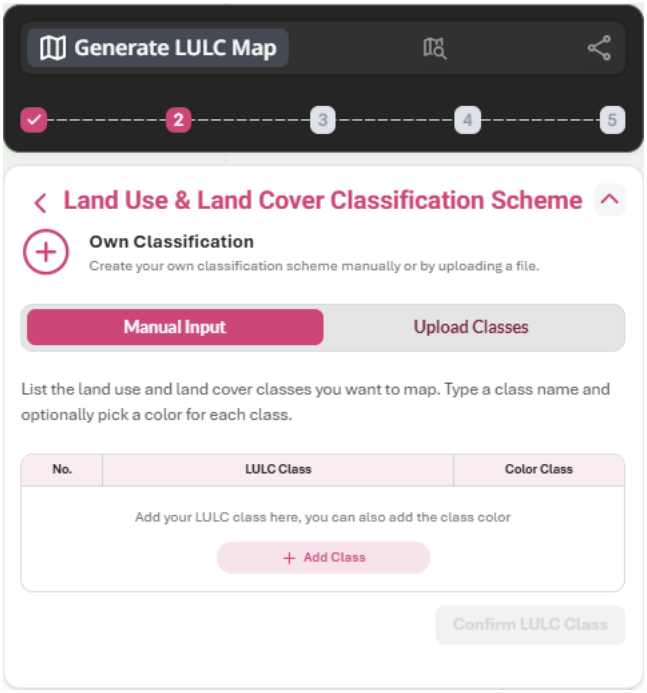
Process

1. The user clicks “Own Classification”.
2. The system shows two options within the Own Classification window: Manual Input and Upload Classes.
3. “Manual Input” and “Upload Classes” panels activated.

Step 2.1.a.1: Manual Input

Overview

The user inputs the land use and land cover classes they want to map. They need to type a class name and optionally pick a colour for each class.



User Input to Interface

1. The user clicks “Add Class” button.
2. The user fills out LULC class name and selects class color.

Process

1. The user enters LULC class name and pick a colour out of 28 colour options.

2. The system stores each class entry individually as it is added.
3. The system validates class ID, name, and checks for uniqueness with the functions in Section 2.1.2.

> **i** Success Notification
>
> - All added classes are displayed in the class recorded table for preview
> - Upload Classes option still available; if clicked, continue to **?@sec-map-generation-upload-classes** Process 1
> - Confirm LULC Class enabled; once clicked, all inputs are locked and a summary table is previewed, along with three options:
>   - "Edit" (pencil icon); continue to Process 4
>   - "Download LULC classes (CSV)"; a `.csv` file containing the inputted classification scheme is downloaded to user's local device.
>   - "Next"; continue to Process 5

> **!** Error Handling Notification
>
> The system returns a validity flag with an error message if invalid

4. If the user edits or deletes the classes in the summary table preview, the system stores each class entry individually as it is added and runs the functions in Section 2.1.2.

> **i** Success Notification
>
> - Each class row can be edited individually
> - New classes can be added with Add Class button
> - If all classes are deleted, back to the beginning of **?@sec-map-generation-lulc-classification**
> - "Done" button appears; if clicked:
>   - "Next" button appears; continue to Process 5
>   - "Download LULC classes (CSV)" appears; a `.csv` file containing the inputted classification scheme is downloaded to user's local device

> **!** Error Handling Notification
>
> The system returns a validity flag with an error message if invalid

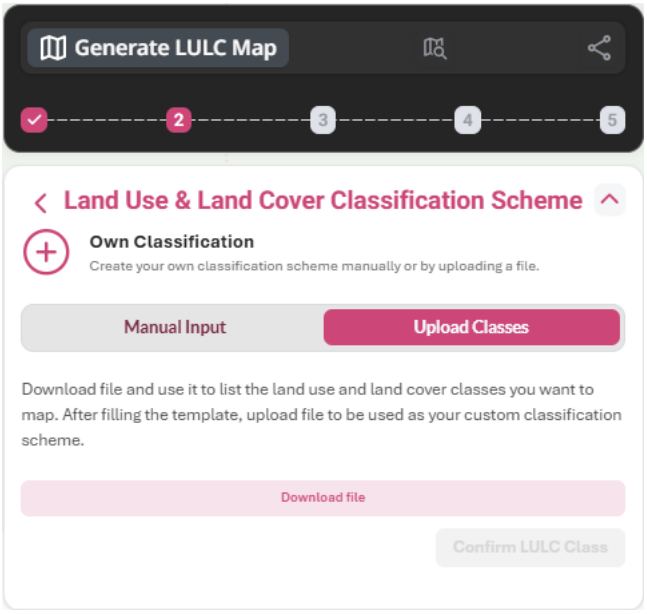
5. If the user clicks on the "Next" button, the user will be directed to **?@sec-map-generation-sample-data**.
6. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-**

satellite-imagery Process 8 is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-lulc-classification**.

Step 2.1.a.2: Upload Classes

Overview

The user downloads a classification template `.csv` file and uses it to list the land use and land cover classes they want to map. After filling the template, they need to upload the file to be used as the custom classification scheme.



User Input to Interface

1. The user clicks "Upload Classes" button.
2. The user uploads the `.csv` file containing the classification scheme information containing at least three columns: a class ID (numeric identifier), a class name, and an optional hex colour code specifying the display colour for each class.

Process

1. The user downloads the `.csv` file containing the classification scheme information template into the user's local device.
2. The user uploads the `.csv` file containing the classification scheme information.
3. The system stores the file uploaded and runs the functions in Section 2.1.1.

Success Notification

If the classification scheme file successfully uploaded:

- A purple box indicating the uploaded file with its file size appears
- A summary table of recorded classes with its ID number and colour class appears
- Manual Input option still available; if clicked, continue to **?@sec-map-generation-manual-input** Process 3
- Confirm LULC Class enabled, if clicked, a summary table is previewed, along with three options:
  - "Edit" (pencil icon); continue to Process 4
  - "Download LUCL classes (CSV)"; a `.csv` file containing the inputted classification scheme is downloaded to user's local device.
  - "Next"; continue to Process 5

Error Handling Notification

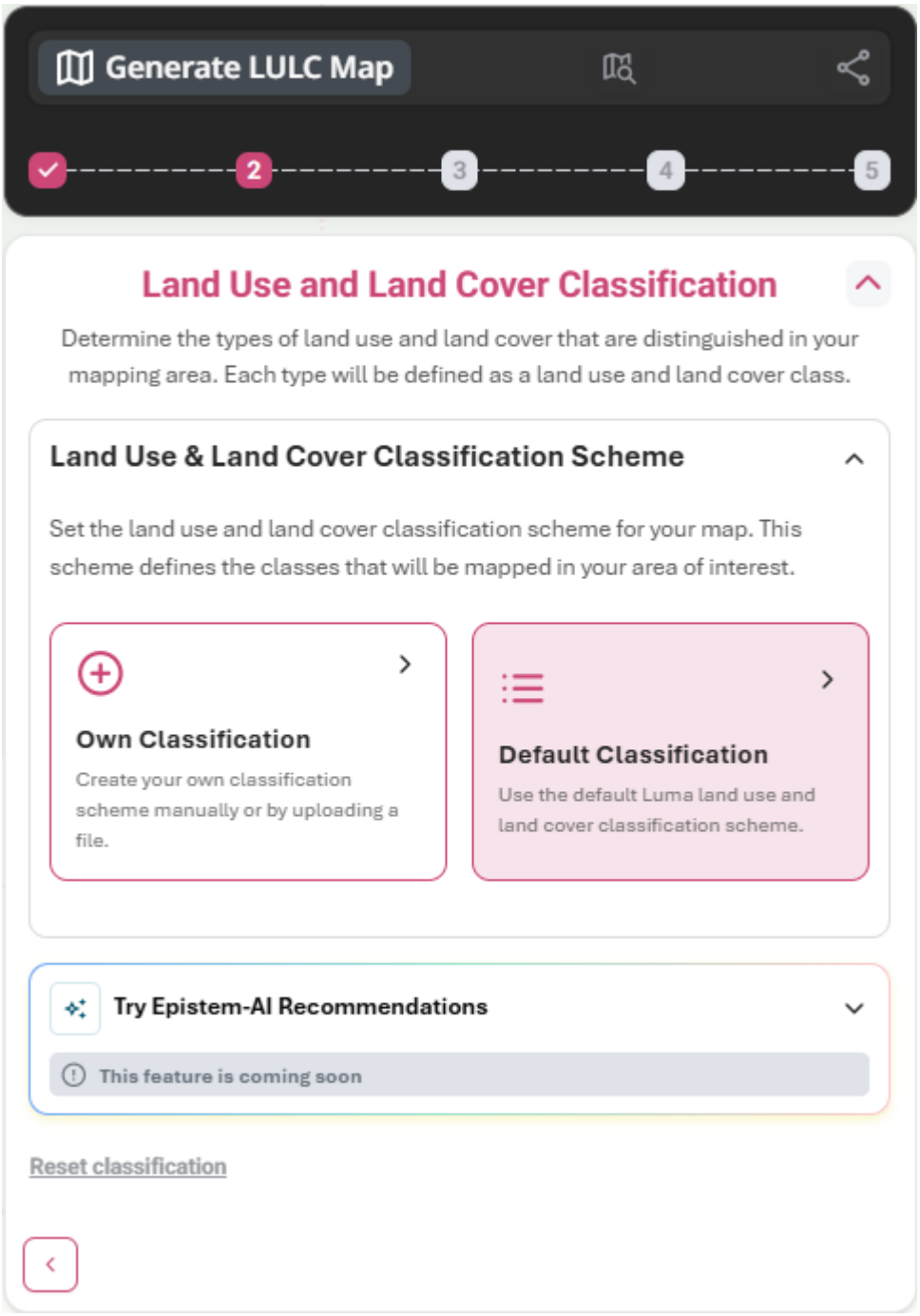
The system returns a validity flag with an error message if invalid

4. If the user clicks on the "Edit" button, each class row can be edited individually. The uploaded file can be deleted, if clicked, all inputs are being reset, back to the beginning of **?@sec-map-generation-lulc-classification**.
5. If the user clicks on the "Next" button, the user will be directed to **?@sec-map-generation-sample-data**.
6. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-satellite-imagery** Process 8 is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-lulc-classification**.

Step 2.1.b: Default Classification

Overview

The user activates class options from default Luma land use and land cover classification scheme. The user is given the option to choose only a specific subset of the default classes, rather than being required to utilize the entire set.



User Input to Interface

1. The user clicks Default Classification to use Epistem classification scheme.
2. The user selects LULC classes from the existing classification scheme.

Process

1. The user selects subsets of the default classes.
2. The classification table is being saved to the system.
3. The system loads the selected classes from a dataset by running functions in Section 2.1.3.

> **Success Notification**
>
> - A summary table of recorded classes with its ID number and colour class appears
> - Two buttons appear:
>
> – "Reset classification"; continue to Process 4
> – "Next"; continue to Process 5

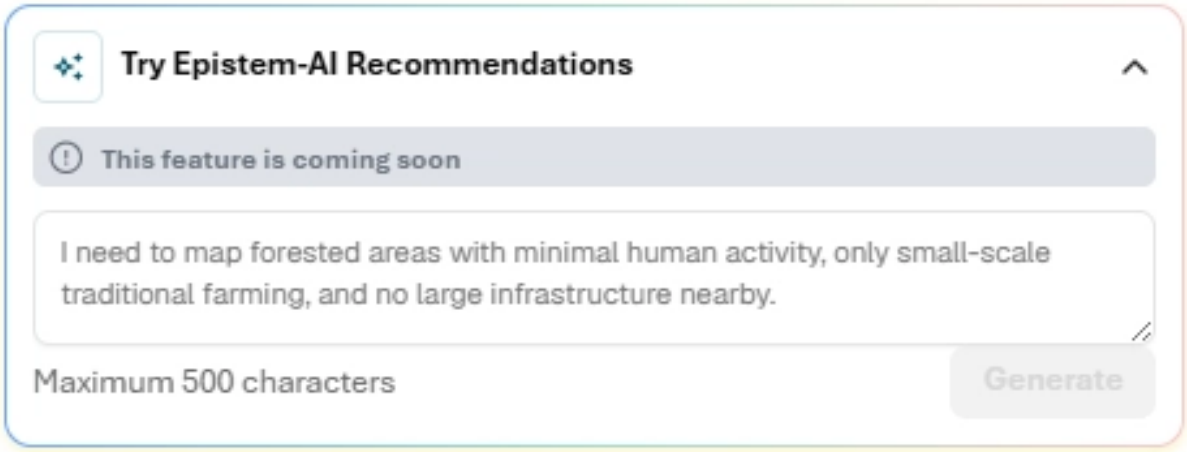
4. If the user clicks on "Reset classification", All inputs are being reset, back to the beginning of **?@sec-map-generation-lulc-classification**.
5. If the user clicks on the "Next" button, the user will be directed to **?@sec-map-generation-sample-data**.
6. If the user clicks on the "Edit" button, all inputs are being reset, back to the beginning of **?@sec-map-generation-lulc-classification**.
7. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-satellite-imagery** Process 8 is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-lulc-classification**.



Step 2.2: Try Epistem-AI Recommendations

Overview

This feature is coming soon.



Step 3: Mapping Sample Data

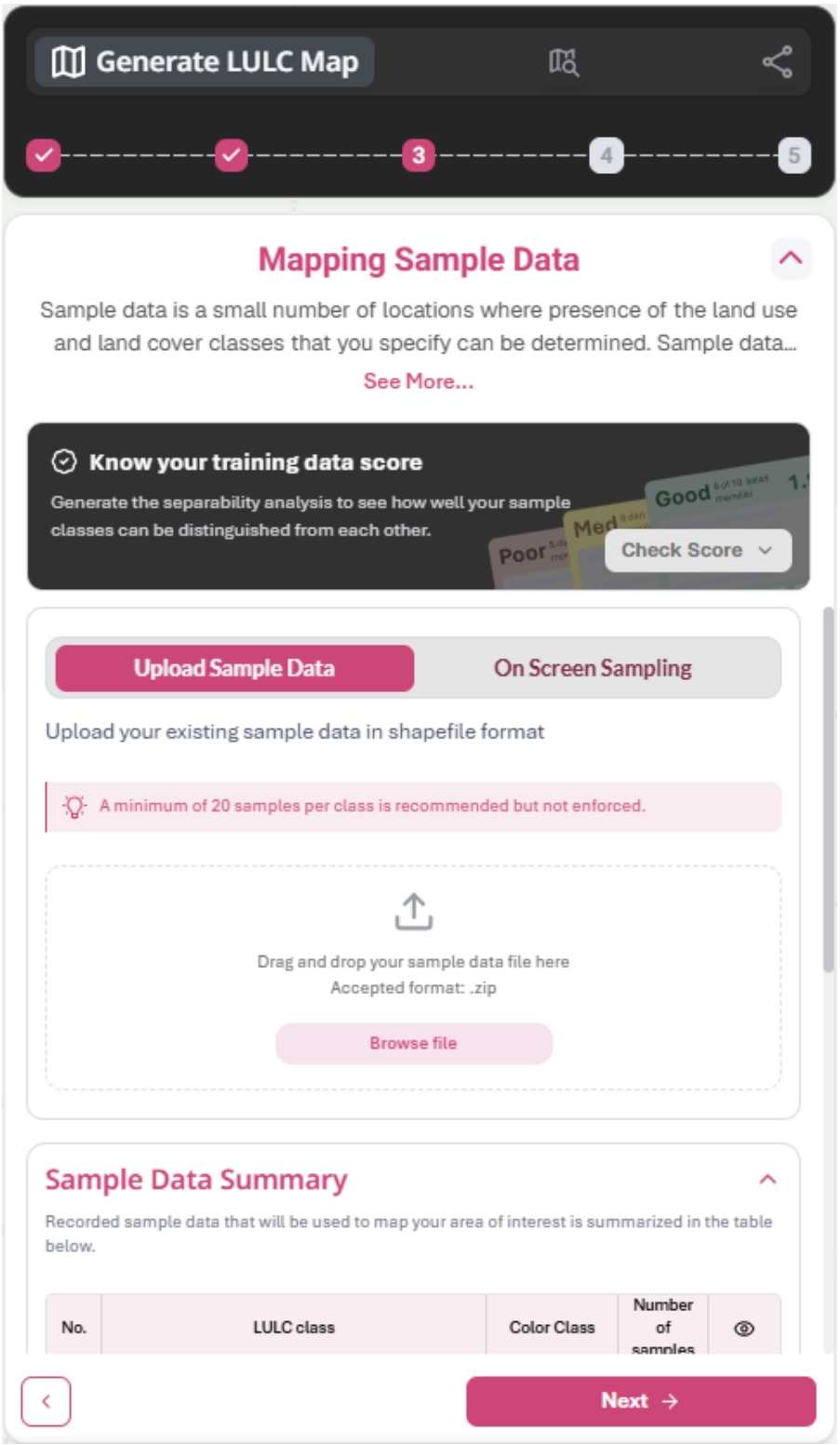
Overview

The user prepares the sample data for land use and land cover classification in their mapping area. The user can upload existing data or create samples directly in Luma through on-screen sampling. In this step the Sample Data Summary selected in the previous step is also shown. The spectral separability of LULC classes is also being assessed in this step using sample data and near-cloud-free satellite imagery. Pairwise class separability is quantified using the Transformed Divergence (TD) method based on pixel-level spectral statistics, using only the spectral data from **?@sec-map-generation-basic-information**.

Step 3.1: Upload Sample Data

Overview

The user uploads their existing sample data in `.zip` file format.


Generate LULC Map
3
4
5
Mapping Sample Data
Sample data is a small number of locations where presence of the land use and land cover classes that you specify can be determined. Sample data...
See More...
Know your training data score
Generate the separability analysis to see how well your sample classes can be distinguished from each other.
Poor
Med
Good
Check Score
Upload Sample Data
On Screen Sampling
Upload your existing sample data in shapefile format
A minimum of 20 samples per class is recommended but not enforced.
Drag and drop your sample data file here
Accepted format: .zip
Browse file
Sample Data Summary
Recorded sample data that will be used to map your area of interest is summarized in the table below.
No.
LULC class
Color Class
Number of samples
Next

User Input to Interface

1. The user uploads a `.zip` file containing their sample data.

Process

1. The user uploads the `.zip` file containing the sample data.
2. The system checks if there are any `.shp` file inside the `.zip` file by running the functions in Section 3.1.
3. The system returns the preview of the uploaded sample data on the map canvas and table of the uploaded sample data.

   > **i** Success Notification
   >
   > - A purple box indicating the uploaded file with its file size appears with a delete button, if clicked, continue to the beginning of **?@sec-map-generation-sample-data**
   > - A summary table of Sample Data Summary with ID number, LULC class name, colour class, number of samples, and buttons to view each class appears

   > **!** Error Handling Notification
   >
   > The system provides an error notification if the system fails to conduct the sample data verification, along with the specific message at which step the validation failed

4. The user clicks on the "Check Score" button to know their sample data quality.
5. The system provides a selection for Spatial Resolution (m) and Maximum Pixels Per Class. The default value is 100 m and 5000 pixels.
6. The system conducts the separability analysis by running the functions in Chapter 4.

   > **i** Success Notification
   >
   > - Training Data Quality result appears, with its category (good, weak, or poor) for each class, and highlights which LULC classes require improvement. A data frame containing details of the pairwise separability is also displayed.
   > - The user can choose several paths:
   >   - Delete the uploaded file; all input in **?@sec-map-generation-upload-sample-data** is deleted and continue to the beginning of **?@sec-map-generation-sample-data**
   >   - On Screen Sampling:
   >   * Single Point Input; continue to **?@sec-map-generation-single-**

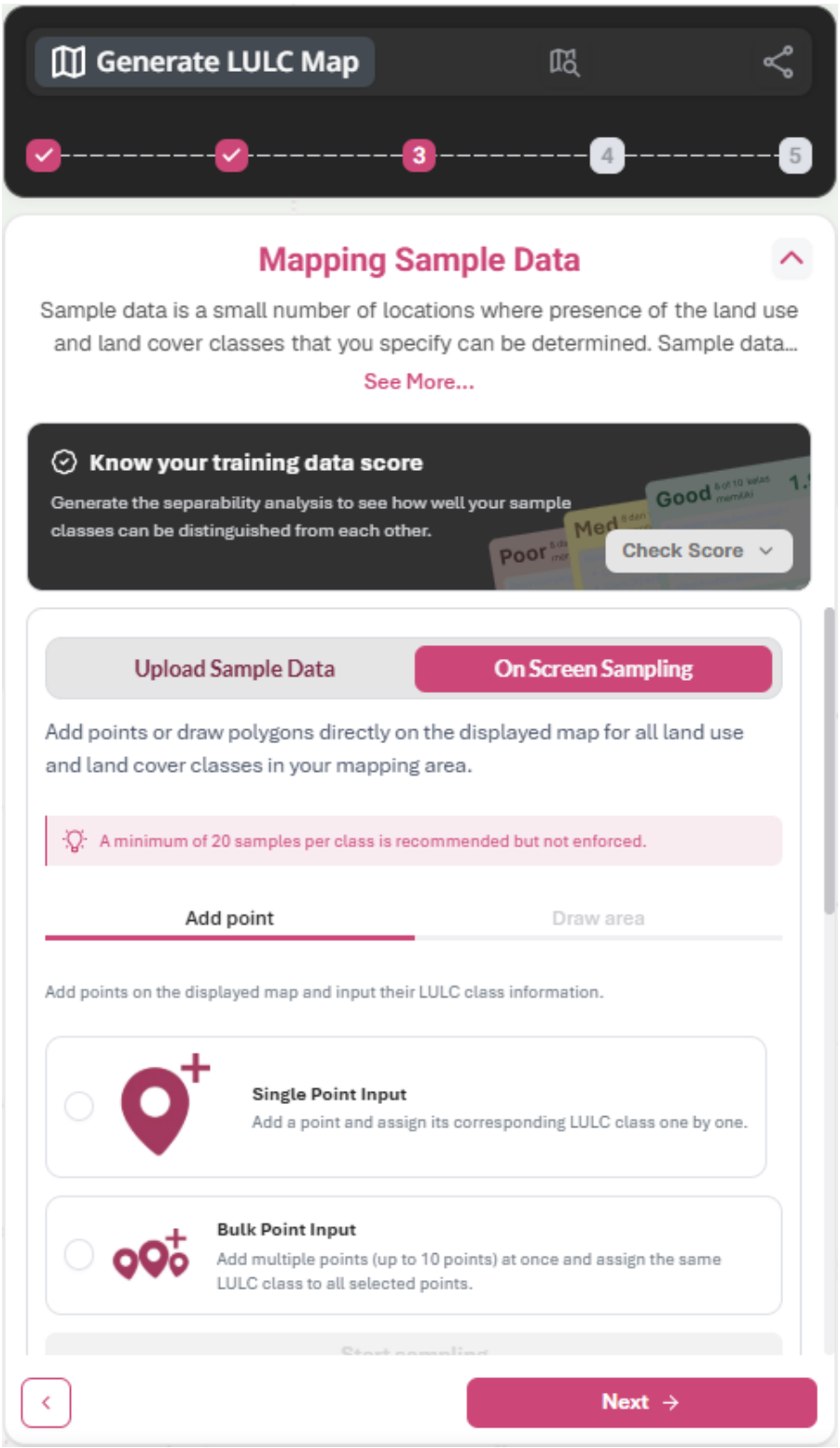
point
    - Bulk Point Input; continue to **?@sec-map-generation-bulk-point**
- Next; continue to **?@sec-map-generation-parameters**

7. If the user clicks on the "Recheck Score" button, continue to Process 5 and 6.
8. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-lulc-classification** summary table of recorded classes is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-sample-data**.

Step 3.2: On Screen Sampling

Overview

The user adds points or draws polygons directly on the interactive displayed map for all land use and land cover classes in their mapping area.


Generate LULC Map
3
4
5
Mapping Sample Data
Sample data is a small number of locations where presence of the land use and land cover classes that you specify can be determined. Sample data...
See More...
Know your training data score
Generate the separability analysis to see how well your sample classes can be distinguished from each other.
Poor
Med
Good
Check Score
Upload Sample Data
On Screen Sampling
Add points or draw polygons directly on the displayed map for all land use and land cover classes in your mapping area.
A minimum of 20 samples per class is recommended but not enforced.
Add point
Draw area
Add points on the displayed map and input their LULC class information.
Single Point Input
Add a point and assign its corresponding LULC class one by one.
Bulk Point Input
Add multiple points (up to 10 points) at once and assign the same LULC class to all selected points.
Next

Step 3.2.a: Add Point

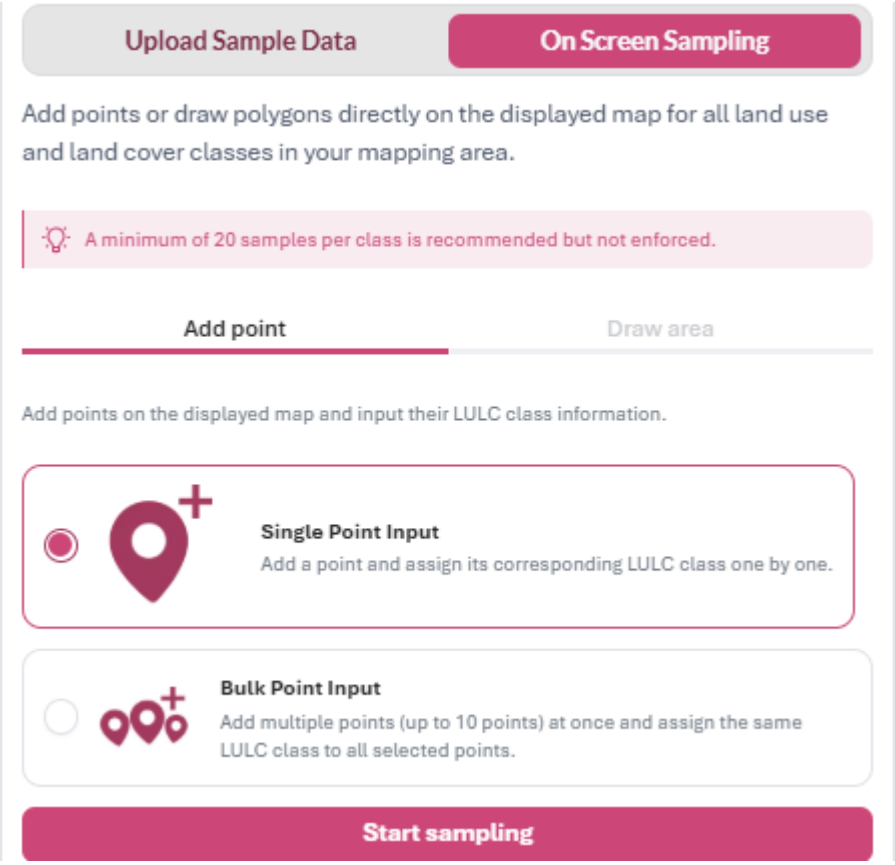
Overview

The user adds points on the interactive displayed map and input their land use and land cover class information.

Step 3.2.a.1: Single Point Input

Overview

The user adds a point and assigns its corresponding land use and land cover class one by one from the drop down list.



User Input to Interface

1. The user assigns new sample points one by one accordingly to the correct class.

Process

1. The user assigns a sample point on the map accordingly to the correct class from the drop down list.
2. The system shows a confirmation that the pinned points have been loaded to the on-screen canvas, along with the number of sample data on the Summary Table.

   > **i** Success Notification
   >
   > - The number of samples in the Summary Table increased based on the user's inputs
   > - End Pointing button enabled; if clicked, continue to Process 3

3. If the user clicks on "End Pointing" button, the user is redirected to **?@sec-map-generation-oss** with Sample Data Summary table and "Training Data Score Check" button enabled.
4. The user clicks on the "Check Score" button to know their sample data quality.
5. The system provides a selection for Spatial Resolution (m) and Maximum Pixels Per Class. The default value is 100 m and 5000 pixels.
6. The system conducts the separability analysis by running the functions in Chapter 4.

   > **i** Success Notification
   >
   > - Training Data Quality result appears, with its category (good, weak, or poor) for each class, and highlights which LULC classes require improvement. A data frame containing details of the pairwise separability is also displayed.
   > - The user can choose several paths:
   >   - Delete the uploaded file; all input in **?@sec-map-generation-upload-sample-data** is deleted and continue to the beginning of **?@sec-map-generation-sample-data**
   >   - On Screen Sampling:
   >   - Single Point Input; continue to **?@sec-map-generation-single-point**
   >   - Bulk Point Input; continue to **?@sec-map-generation-bulk-point**
   >   - Upload Sample Data; continue to **?@sec-map-generation-upload-sample-data**
   >   - Next; continue to **?@sec-map-generation-parameters**

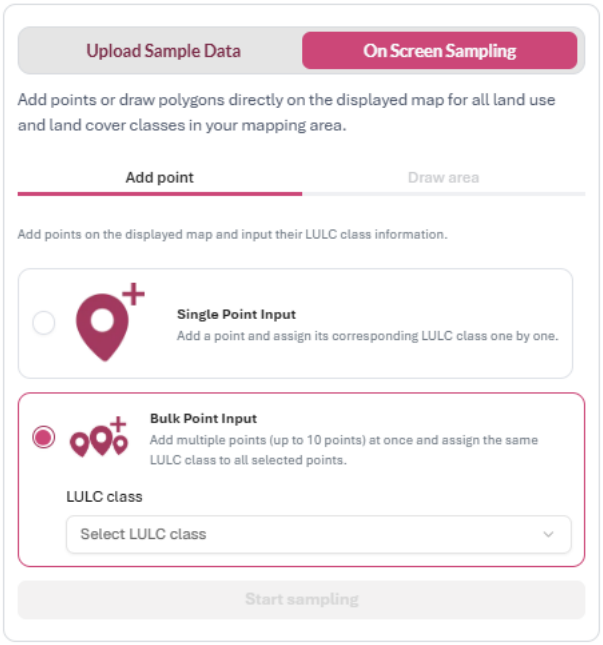
7. If the user clicks on the "Recheck Score" button, continue to Process 5 and 6.
8. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-lulc-classification** summary table of recorded classes is being shown. The user has to

click the “Next” button to go back to the last edited panel in **?@sec-map-generation-sample-data**.

Step 3.2.a.2: Bulk Point Input

Overview

The user adds multiple points (up to 10 points) at once and assigns the same land use and land cover class to all selected points.



User Input to Interface

1. The user assign new sample points accordingly to the one class at a time.

Process

1. The user chooses a class from the drop down list and assigns sample points on the map accordingly to the correct class.
2. The system shows a confirmation that the pinned points have been loaded to the on-screen canvas, along with the number of sample data on the Summary Table.

> **Success Notification**
>
> - The number of samples in the Summary Table increased based on the user's inputs
> - End Pointing button enabled; if clicked, continue to Process 3

3. If the user clicks on "End Pointing" button, the user is redirected to **?@sec-map-generation-oss** with Sample Data Summary table and "Training Data Score Check" button enabled.
4. The user clicks on the "Check Score" button to know their sample data quality.
5. The system provides a selection for Spatial Resolution (m) and Maximum Pixels Per Class. The default value is 100 m and 5000 pixels.
6. The system conducts the separability analysis by running the functions in Chapter 4.

> **Success Notification**
>
> - Training Data Quality result appears, with its category (good, weak, or poor) for each class, and highlights which LULC classes require improvement. A data frame containing details of the pairwise separability is also displayed.
> - The user can choose several paths:
>   - Delete the uploaded file; all input in **?@sec-map-generation-upload-sample-data** is deleted and continue to the beginning of **?@sec-map-generation-sample-data**
>   - On Screen Sampling:
>   - Single Point Input; continue to **?@sec-map-generation-single-point**
>   - Bulk Point Input; continue to **?@sec-map-generation-bulk-point**
>   - Upload Sample Data; continue to **?@sec-map-generation-upload-sample-data**
>   - Next; continue to **?@sec-map-generation-parameters**

7. If the user clicks on the "Recheck Score" button, continue to Process 5 and 6.
8. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-lulc-classification** summary table of recorded classes is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-sample-data**.



Step 3.2.b: Draw Area

Overview

This feature is coming soon.

Step 3.3: When user used Default Classification in Step 2

Overview

The user who chose the subsets of default classification from **?@sec-map-generation-default-classes** does not need to add sample data.

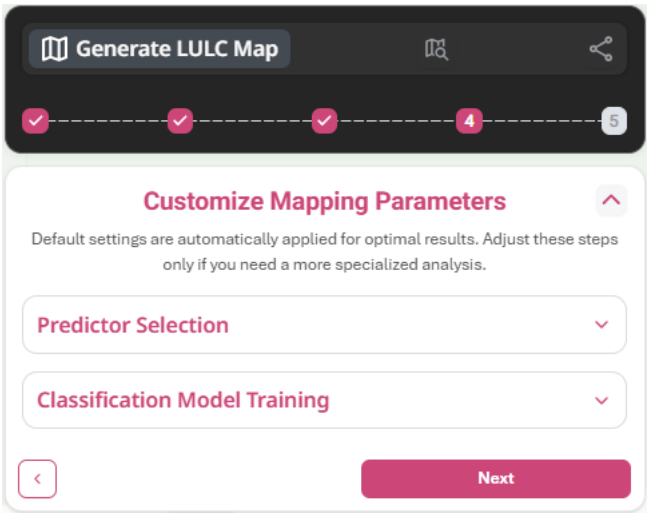
Process

1. The system shows the Sample Data Summary table with number of samples available from default training data and shows pins of the related class on the map canvas.
2. The “Next” button is activated.
3. If the user clicks “Next”.
4. The system loads the next step.

Step 4: Customize Mapping Parameters

Overview

The user can choose a set of predictors that optimize the discrimination of LULC classes in the classification model. Predictors are generated from three principal categories: spectral transformations (indices), terrain-based metrics, and distance-based metrics, capturing complementary spectral, topographic, and spatial information.



Step 4.1: Predictor Selection

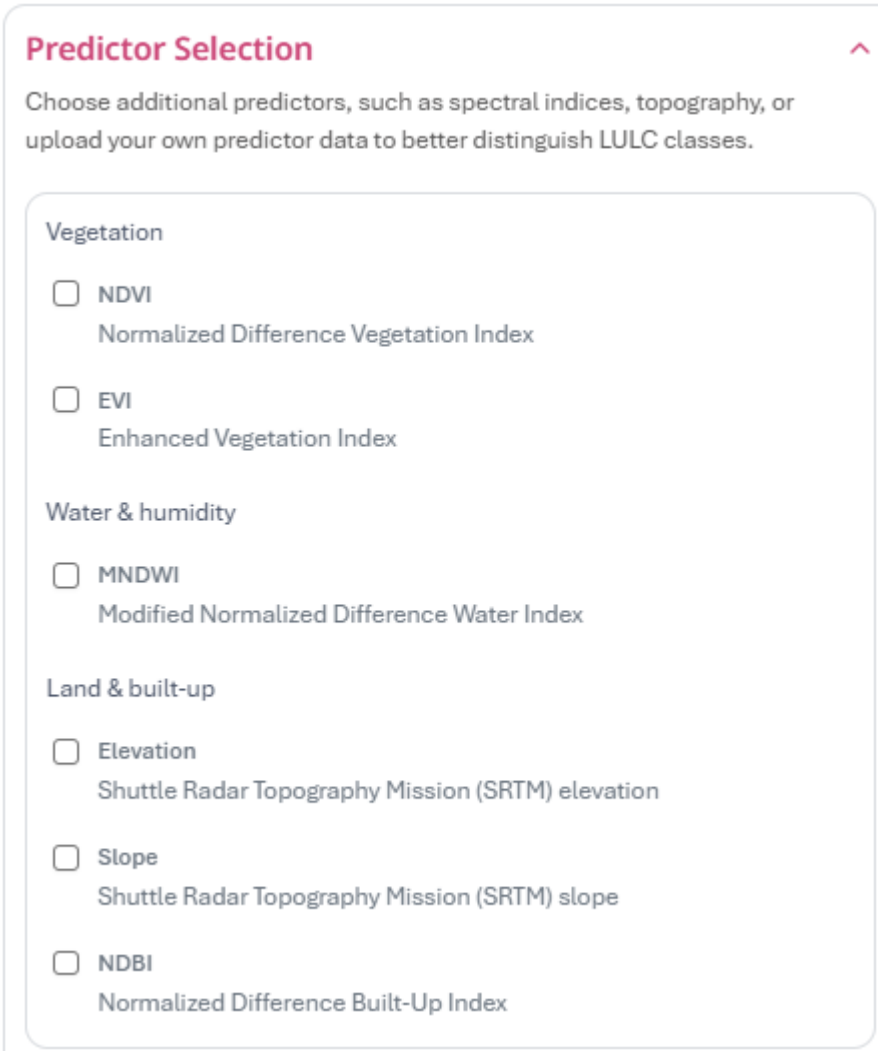
Overview

The user is prompted to select the predictors currently supported by Luma Geospatial Engine by using check boxes or upload their own predictor data in `.zip` file.

Step 4.2.a: Default Predictors

Overview

The user selects the predictors currently supported by Luma Geospatial Engine by using check boxes.



User Input to Interface

1. The user selects the predictors by using check boxes.

Process

1. The user selects the predictors from the list.
2. The system retrieves the predictors based on the user's selection and runs the functions in Section 5.1.

> **Success Notification**
>
> - "Next" button leads to Summary of LULC List Parameters showing the predictors selected and Random Forest Variable in a purple box.
> - Two buttons appear:
>   - "Change Input"; continue to Process 3
>   - "Next"; continue to Process 4

3. The user clicks on "Change Input".

> **Success Notification**
>
> - "Change Input"; all inputs being reset and back to the beginning of **?@sec-map-generation-parameters**
> - "Discard Changes"; stay in Process 3 with "Cancel Change" button activated

4. If the user clicks on "Next" button, the system generates map with all input from Step 1-4.

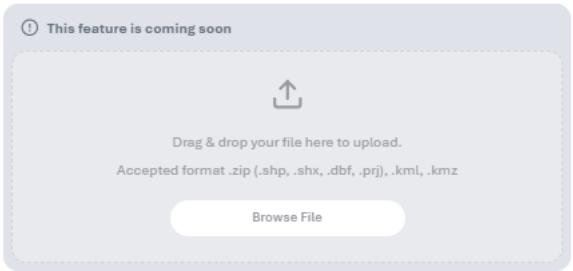
> **Success Notification**
>
> A pop up window appears with User's Input Summary and Summary of LULC Class Compositions in purple boxes, with a "Generate Map" button. Continue to **?@sec-map-generation-final**.

5. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-sample-data** Sample Data Summary is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-parameters**.

Step 4.2.b: Upload Predictors

Overview

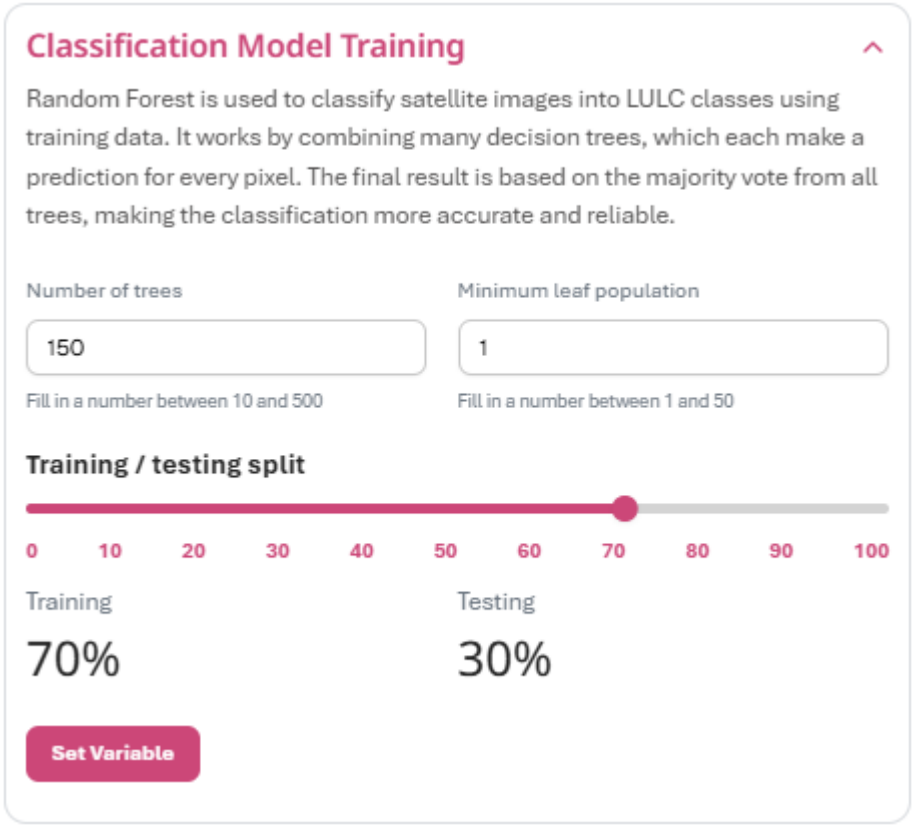
This feature is coming soon.



Step 4.2: Classification Model Training

Overview

The user is given the option to adjust the classification model by adjusting the value of the Random Forest variable to classify the satellite images into land use and land cover classes using training data. The default value of the Random Forest parameters are 150 number of trees and 1 minimum leaf population. The user is also prompted to define the proportion to split the sample data into training and validation dataset. The default value of the dataset proportion is 70% for training data and 30% for validation data.



User Input to Interface

1. The user configures the classification parameters for number of trees and minimum leaf population.
2. The user defines the proportion to split the sample data into training and validation dataset.

Process

1. The user sets the Random Forest parameters and Training/Testing Split.
2. The system verifies the inputs and runs the process to create the classification model in Section 6.2 and splits the sample data with functions in Section 6.1.1.

   > **Success Notification**
   >
   > "Set Variable" button leads to Summary of LULC List Parameters showing the predictors selected and Random Forest Variable in a purple box. Two buttons appear:
   >
   > - "Change Input"; continue to Process 3
   > - "Next"; continue to Process 4

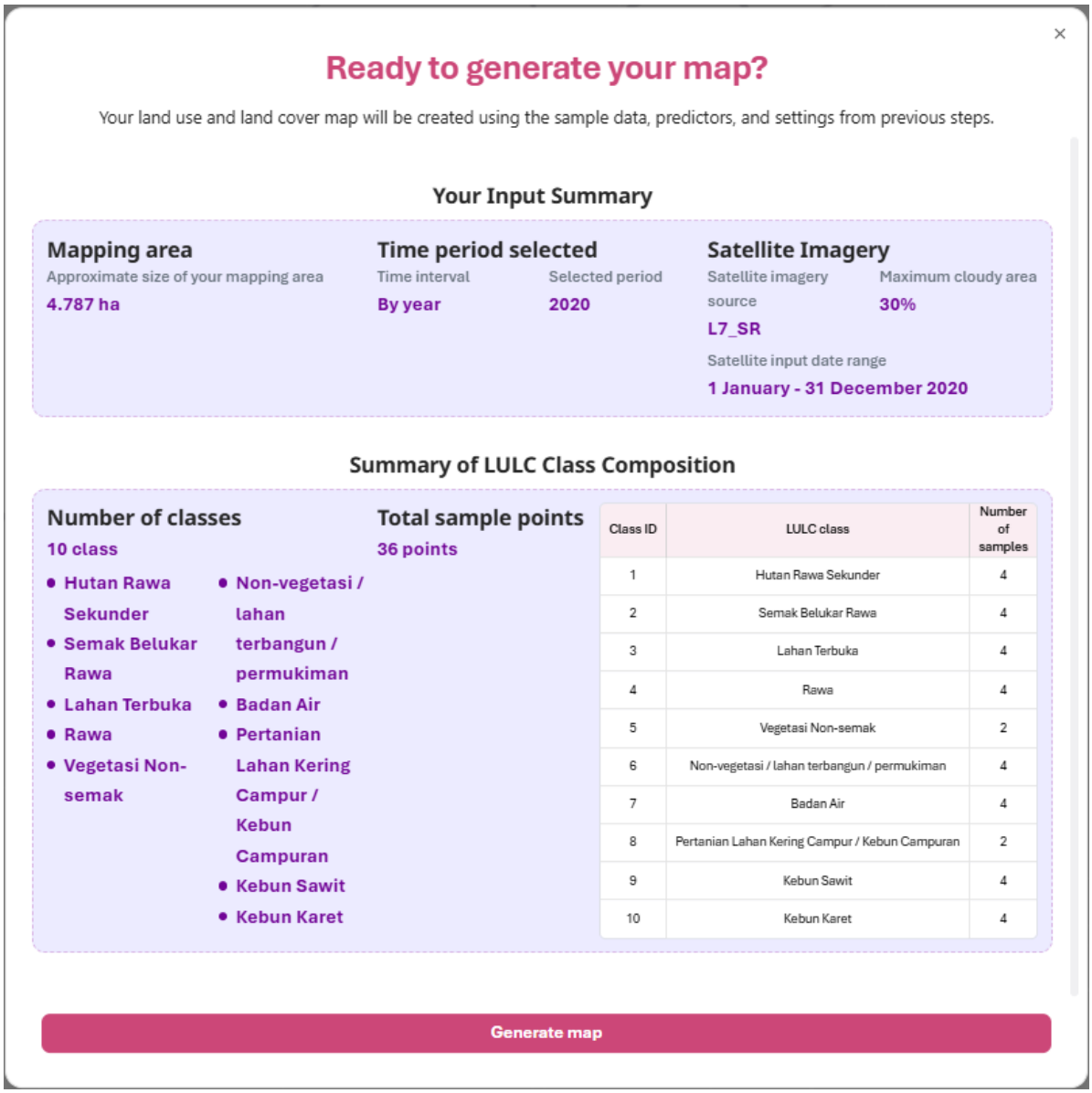
3. The user clicks on "Change Input".

   > **Success Notification**
   >
   > "Edit" (pencil icon) appears next to each parameter; if clicked, a pop up window appears asking if user wants to:
   >
   > - "Change Input"; all inputs being reset and back to the beginning of **?@sec-map-generation-parameters**
   > - "Discard Changes"; stay in Process 3 with "Cancel Change" button activated

4. If the user clicks on "Next" button, the system generates map with all input from Step 1-4.

   > **Success Notification**
   >
   > A pop up window appears with User's Input Summary and Summary of LULC Class Compositions in purple boxes, with a "Generate Map" button. Continue to **?@sec-map-generation-final**.

5. If the user clicks on the "Back" (chevron) button, the panel of **?@sec-map-generation-sample-data** Sample Data Summary is being shown. The user has to click the "Next" button to go back to the last edited panel in **?@sec-map-generation-parameters**.



Step 4.3: When user used Default Classification in Step 2

Overview

The user who chose the subsets of default classification from **?@sec-map-generation-default-classes** does not need to customize the mapping parameters.

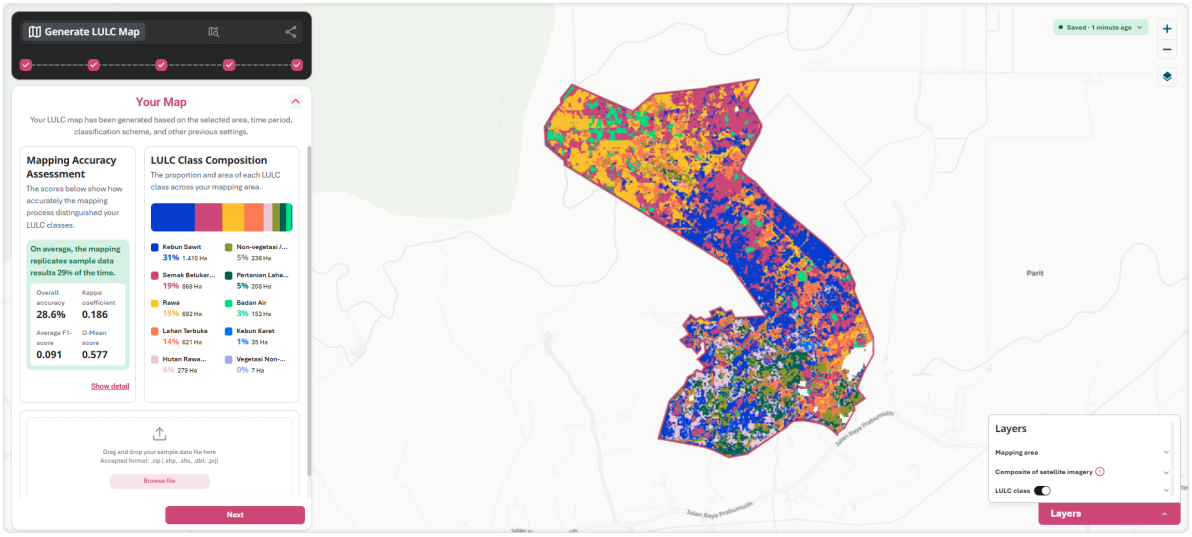
Process

1. The system shows disabled mapping parameters.
2. The “Next” button is activated.
3. If the user clicks “Next”.
4. The system loads the next step.
5. Continue to **?@sec-map-generation-final**.

Step 5: Map Generation

Overview

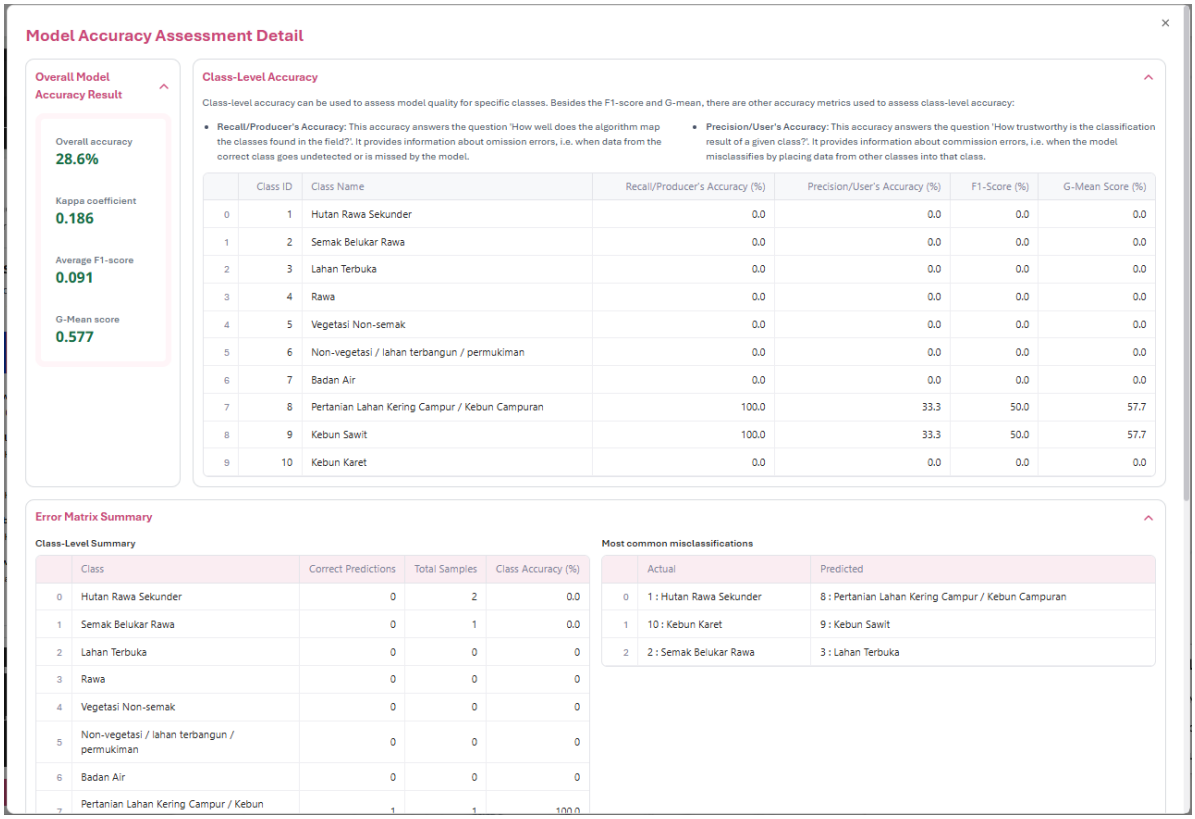
The user is shown the map generated on the map canvas with the classes in different colours, and the panel is showing model accuracy results, LULC class composition, and map's thematic accuracy assessment process.



Step 5.1: Model Accuracy Assessment

Overview

The user is shown how accurately the mapping process distinguished their land use and land cover classes.



User Input to Interface

1. The user clicks on “Show Detail”.

Process

1. The user clicks on “Show Detail”.
2. The system retrieves the mapping accuracy assessment result.

> **Success Notification**
>
> A pop up window shows:
>
> - Overall Model Accuracy Result
>   - Overall accuracy
>   - Kappa coefficient
>   - Average F1-score
>   - G-mean score
> - Class Level Accuracy

  - Recall/Producer's accuracy
  - Precision/User's accuracy
- Error Matrix Summary
  - Class-level summary
  - Most common misclassifications
- Confusion Matrix

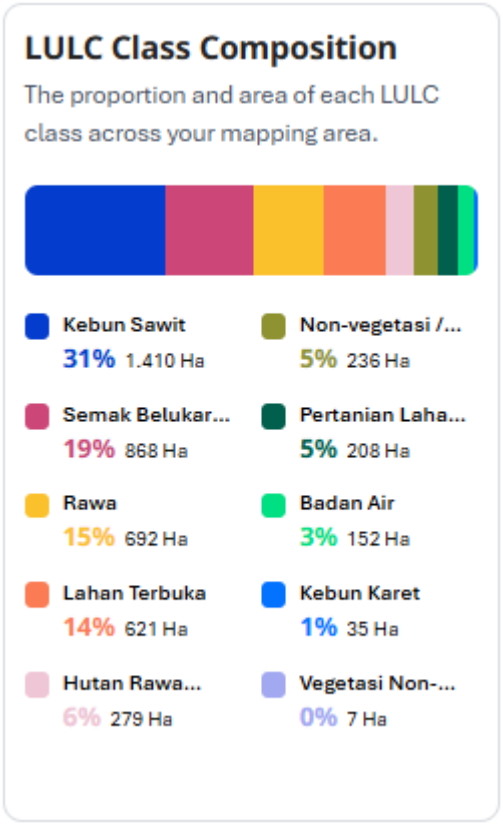
Two buttons appear at the bottom:

- "Download Raw Data"; `.csv` file containing the raw data is downloaded to user's local device
- "Download Chart"; `.png` file of the confusion matrix is downloaded to user's local device

Step 5.2: LULC Class Composition

Overview

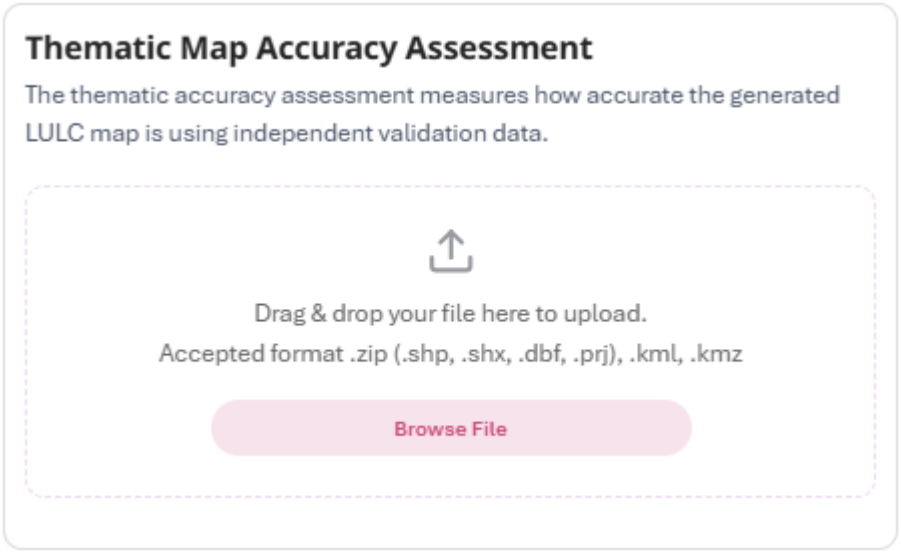
The user can visualize the LULC classification result on the map canvas along with the training data layer and is shown the proportion of each LULC class.



Step 5.3: Thematic Accuracy Assessment

Overview

The user uploads an independent ground reference data in `.zip` file to perform a comprehensive thematic accuracy assessment of the land cover map.



User Input to Interface

1. The user uploads validation data in a `.zip` file.

Process

1. The user uploads `.zip` file containing the ground reference data.
2. The system verifies if the `.shp` of the validation map is the uploaded inside the `.zip` file and runs the functions in Section 7.2.

> **Success Notification**
>
> A pink box showing the Thematic Accuracy Result appears with overall prediction, Kappa coefficient, number of points correct and wrong. Two actions can be chosen:
>
> - "Reupload validation data"
> - "Show detail"; continue to Process 4

> **Error Handling Notification**
>
> The system shows an error message if the system fails to run the thematic accuracy assessment

3. If the user clicks on "Reupload validation data", continue to the beginning of **?@sec-map-generation-final**.

4. If the user clicks on “Show detail”, the system previews the comprehensive report of the map’s thematic quality.

> **Success Notification**
>
> A pop up window shows:
>
> - Overall Model Accuracy Result
> - Overall accuracy
> - Kappa coefficient
> - Average F1-score
> - G-mean score
> - Class Level Accuracy
> - Recall/Producer’s accuracy
> - Precision/User’s accuracy
> - Error Matrix Summary
>   - Class-level summary
>   - Most common misclassifications
> - Confusion Matrix
>
> Two buttons appear at the bottom:
>
> - “Download Raw Data”; `.csv` file containing the raw data is downloaded to user’s local device.
> - “Download Chart”; `.png` file of the confusion matrix is downloaded to user’s local device.

5. If the user clicks on the “Next” button.

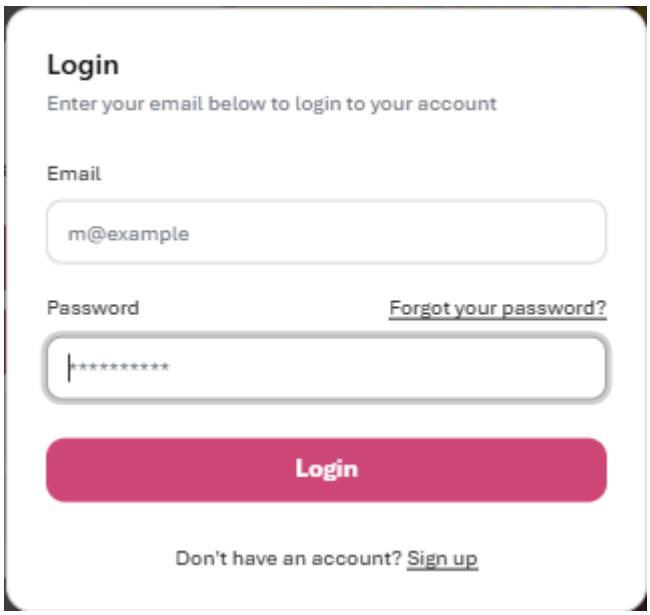
> **Success Notification**
>
> A pop up window shows several options:
>
> - “Download map”; user is prompted to log in/sign up, continue to Feature 2
> - “Share map”; user is prompted to log in/sign up, continue to Feature 3

- “Save to my account” (currently disabled)
- “Improve accuracy”

Additional Features

Overview

The user can save their land use and land cover mapping progress and share their mapping results by logging in/signing up to an Epistem account. Once logged in, user account is activated on the upper right corner of the screen with the user's initial or profile picture.



Feature 1: Save Progress

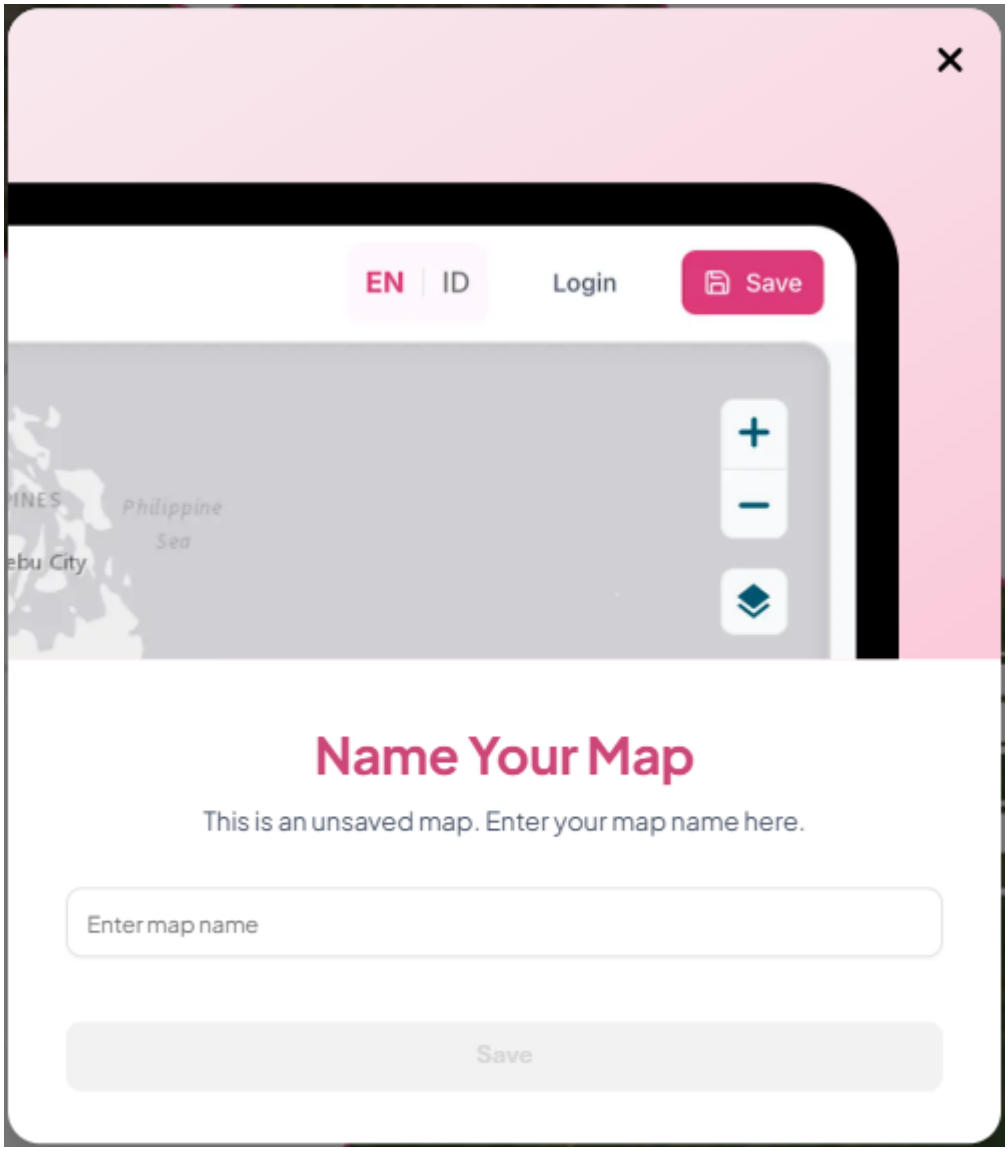
Overview

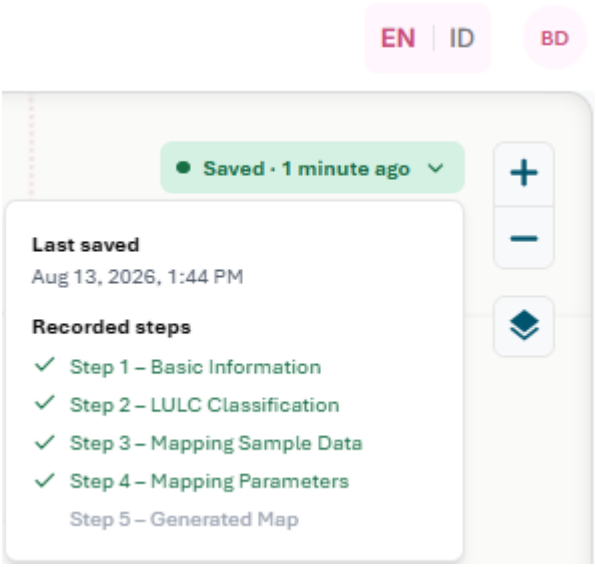
Once logged in, the user can save their land use and land cover mapping progress, name the map they are working on, and view which steps they have done and where they are currently in. This can be viewed on the upper right corner of the map canvas. The session is stored for up to 24 hours within the database.



Process

1. The user clicks “Save” button.
2. The user is prompted to log in or sign up for an Epistem account if they have not already registered.
3. Once logged in, the user can name the map they are working on in order to save it.
4. Progress is saved automatically; completed steps and the user’s current position are visible in the top-right corner of the map canvas.

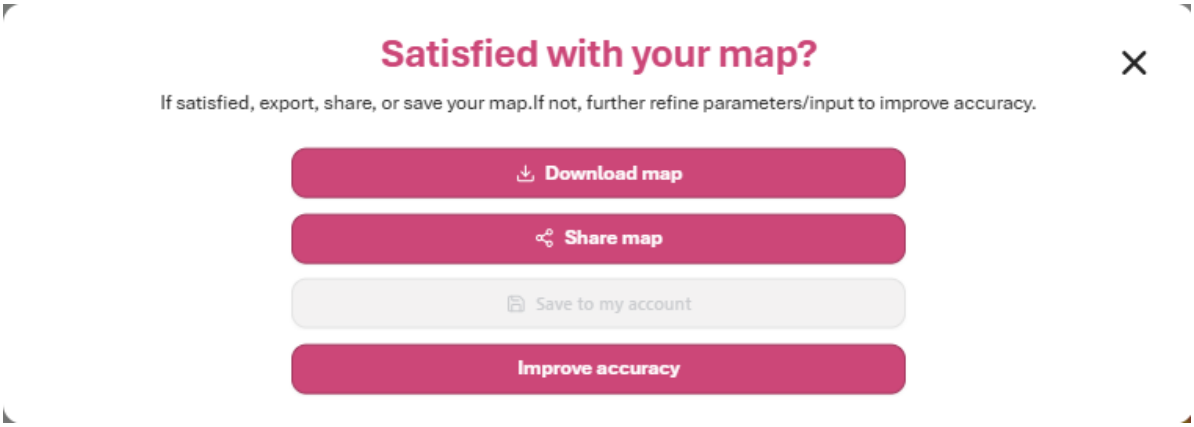




Feature 2: Download Map

Overview

Once logged in, the user can download their generated map.



Process

1. The user downloads their map.
2. The system sends an email containing the exported LULC map with a download link.

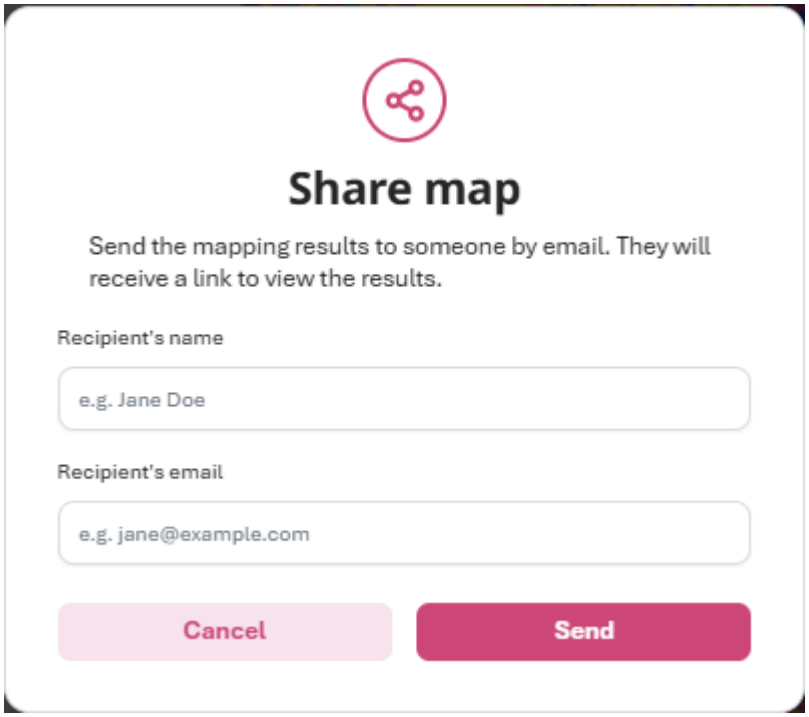
Success Notification

The user receives an email from webdev@epistem.io containing a download link to the GeoTIFF of the exported map

Feature 3: Share Map

Overview

Once logged in, the user can share their generated map to another person with an email address.

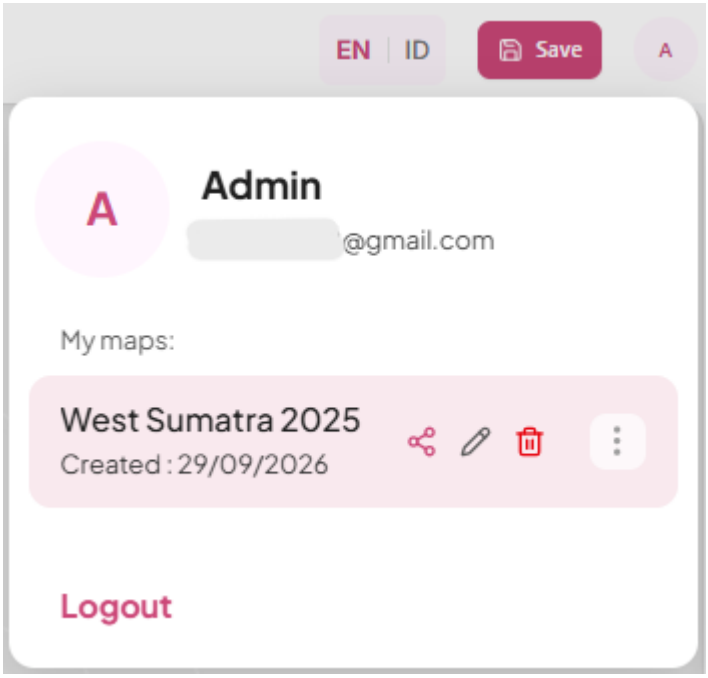


Process

1. The user input recipient's name and email address.
2. The system sends and email to the recipient containing the exported LULC map with a download link.

Success Notification

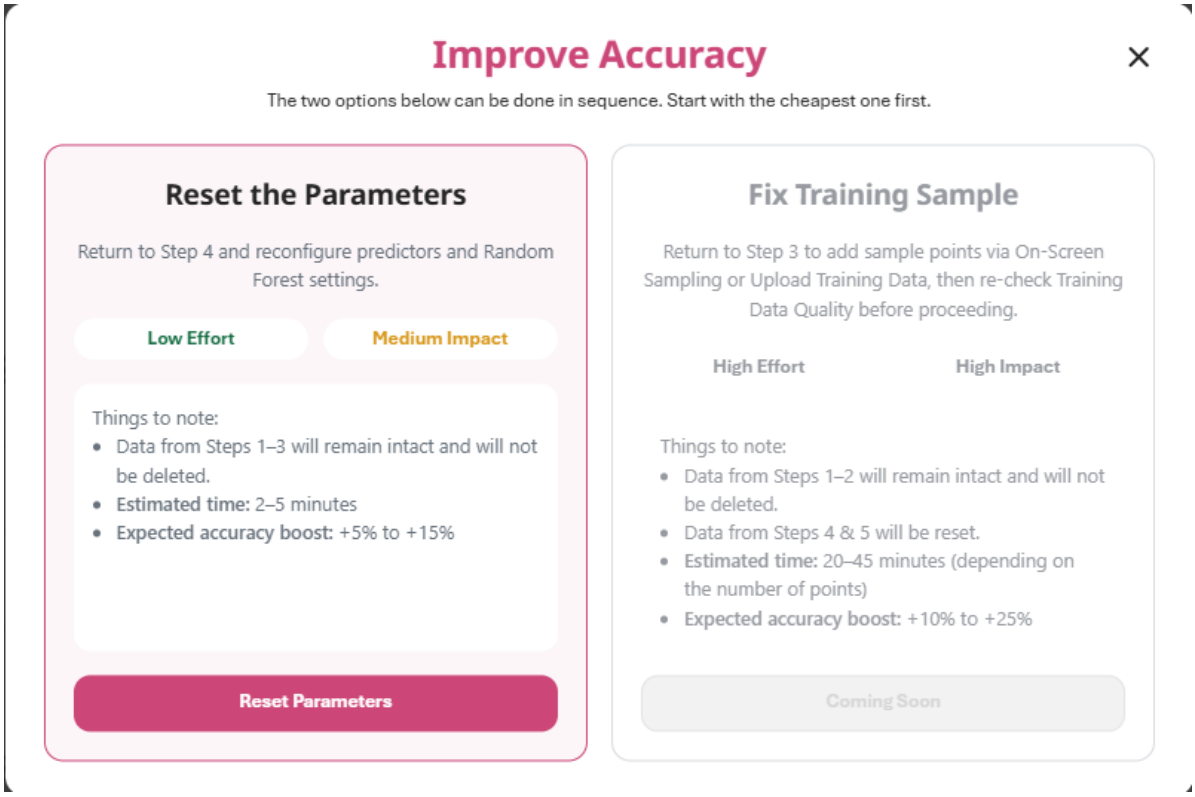
The recipient receives an email from webdev@epistem.io containing a download link to the GeoTIFF of the exported map



Feature 4: Improve Accuracy

Overview

The user can improve the accuracy of their generated map in two ways: by resetting the parameters in **?@sec-map-generation-parameters**, or by adding or editing the sample data in **?@sec-map-generation-sample-data**.



Process

1. The user clicks on “Improve Accuracy”.

   > **i** Success Notification
   >
   > A pop up window showing two methods of improving accuracy:
   >
   > - ”Reset Parameters”; continue to Process 2
   > - ”Fix Training Data”; continue to Step 3 (coming soon)

2. If the user clicks on ”Reset Parameters”, the system retrieves information from Step 4.

   > **i** Success Notification
   >
   > The panel returns to the beginning of **?@sec-map-generation-parameters**, with all selected predictors still checked, and displays a message explaining that the user is in Improvement Mode.

2 Change Analysis

3 Collaborative Mapping

Meijer, J. R., M. A. J. Huijbregts, K. C. G. J. Schotten, and A. M. Schipper. 2018. "Global Patterns of Current and Future Road Infrastructure." *Environmental Research Letters* 13 (6): 064006. https://doi.org/10.1088/1748-9326/aabd42.

Montero, David, César Aybar, Miguel D. Mahecha, Francesco Martinuzzi, Maximilian Söchting, and Sebastian Wieneke. 2023. "A Standardized Catalogue of Spectral Indices to Advance the Use of Remote Sensing in Earth System Research." *Scientific Data* 10: 157. https://doi.org/10.1038/s41597-023-02096-0.

Mullissa, Adugna, Andreas Vollrath, Christelle Odongo-Braun, et al. 2021. "Sentinel-1 SAR Backscatter Analysis Ready Data Preparation in Google Earth Engine." *Remote Sensing* 13 (10): 1954. https://doi.org/10.3390/rs13101954.